01 — DATABASE (current state, One Local / OL-ui)

Scope: this repository only (`OL-ui`: customer app `frontend/`, restaurant app `partner/`, server code in `frontend/server/` served through Expo Router API routes `frontend/app/api/`). Nothing was modified; nothing was run against any database.

Conventions used in every audit file - (no tag) = stated in a file of this repository. - **INFERRED** = deduced from code behaviour, not written down anywhere. - **UNKNOWN** = cannot be determined from this repository. - **DRAFT** = a migration file that the repo's own docs say is applied nowhere (or only on a local database). - **DOC-CLAIM** = the repo's docs/journal say so, but nothing in code or SQL proves it.

0. Where the schema comes from, and what is applied

Source	What it is	Status
`db/migrations/0021_one_local_core.sql` … `0029_ol_dispatch.sql`	The only schema files this repo owns. Prefix rule: every table is `ol_*` except the 0029 delivery tables.	See below
`db/local/0000_os_replica.sql`	Rebuilt copy of the OS/POS tables One Local touches, generated by `scripts/db.py replica` from production's catalog. Local development only. Columns/constraints/indexes only — **no policies**	Local only
`docs/CENTRAL_DB_MAP.md`	Read-only snapshot of production on 2026-09-22 (72 tables, 140 policies, 29 functions). Written *before* 0021 was applied, so its `ol_*` section describes the superseded first draft	DOC-CLAIM, stale
`supabase/config.toml`	Local Supabase config (`project_id = "onelocal-dev"`, Postgres 17, realtime enabled, storage enabled)	Local only
`db/cleanup/2026-09-27_remove_one_local_demo_catalogue.sql`	One-off delete of 237 demo catalogue rows	Run by the founder per journal (DOC-CLAIM)
`db/tests/0029_dispatch_scenario.sql`	SQL scenario for the dispatch functions	Test only

Applied state (from `docs/JOURNAL.md`, **DOC-CLAIM**, cannot be verified from code): 0021–0024 live in production project `ywylhzcxuoeilygzdonx`; 0025–0029 are drafts (0025 and 0026 are applied to the *local* database; 0029 was tried locally and removed). The server code is written to run before those drafts exist (`checkoutsReady()` in `frontend/server/store.ts`, `listingColumnsReady()` in `frontend/server/marketplace.ts`, `tableReady()` in `frontend/server/privacy.ts`).

Migration ledger: `public.schema_migrations (name, applied_at)` belongs to the OS repository (`loki1514/Repo-1-`); the numbers 0025–0029 in this repo are placeholders per the file headers.

There are **no enums** defined by this repo (every "enum" is a `text` column with a `CHECK` list). The OS replica defines Postgres enums `org_status`, `org_type`, `member_status`, `lifecycle_stage`, `onboarding_stage` (not owned here).

---

1. One Local tables (`ol_*`)

1.1 Catalogue tables — migration 0021 (+ audit columns 0024)

All have `RLS ON`, no policies, no grants for `anon`/`authenticated`, full grants for `service_role`.

Audit columns added by 0024 to `ol_service_areas, ol_categories, ol_brands, ol_stores, ol_products, ol_banners, ol_discover_chips, ol_coupons, ol_events, ol_reels, ol_themed_stores`: `created_by uuid → auth.users(id) ON DELETE SET NULL`, `updated_by uuid → auth.users(id) ON DELETE SET NULL`, `deleted_at timestamptz` (soft delete). `ol_media_assets` did not get them.

Table	PK	Columns (type, default, constraints)	FKs	Indexes / uniques
`ol_service_areas`	`city_id text`	name text NN; lat/lng double precision NN; radius_km numeric(6,2) NN CHECK >0; active bool NN default true; created_at	–	PK only. Seed row `('latur','Latur',18.4088,76.5604,15)`
`ol_categories`	`id text`	name NN; emoji NN ''; color; department NN; image NN ''; sort_order int NN 0; is_demo bool NN true; created_at	–	–

Table	PK	Columns (type, default, constraints)	FKs	Indexes / uniques
`ol_brands`	`id text`	name NN; department; logo NN ''; image NN ''; tagline NN ''; city_id NN 'latur'; is_demo; created_at	city_id → ol_service_areas	–
`ol_stores`	`id text`	name NN; logo; tagline; delivery_time text; rating numeric(3,2) NN 4.5 CHECK 0..5; tags text[] NN '{}'; department; media jsonb NN '[]'; organization_id uuid; city_id NN 'latur'; is_demo; sort_order; created_at. **0028 adds:** status text NN 'live' CHECK `draft/pending/live/paused (ol_stores_status_check)`; location_id uuid; cover_image text; lat/lng double precision; delivery_radius_km numeric(5,2); approved_by uuid; approved_at; updated_at NN now()	organization_id → organizations(id) SET NULL; city_id → ol_service_areas; 0028: location_id → locations(id) SET NULL; approved_by → auth.users SET NULL	0028: `ol_stores_one_live_listing` UNIQUE (organization_id, coalesce(location_id, zero-uuid)) WHERE status='live' AND deleted_at IS NULL AND organization_id IS NOT NULL; `ol_stores_city_status (city_id,status)`; trigger `ol_stores_updated_at`
`ol_products`	`id text`	name NN; weight NN ''; price numeric(10,2) NN CHECK ≥0; mrp NN CHECK ≥ price; image NN ''; category_id; store_id text (**no FK**); brand_id text (**no FK**); department NN; delivery_min int NN 15; discount_pct int; rating numeric(3,2) NN 4.5; is_available bool NN true; menu_item_id uuid; city_id NN 'latur'; is_demo; sort_order; created_at	category_id → ol_categories; menu_item_id → menu_items(id) SET NULL; city_id → ol_service_areas	`ol_products_city_department (city_id, department)`, `ol_products_store (store_id)`
`ol_banners`	`id text`	title NN; subtitle; image; bg; city_id NN; is_demo; sort_order; created_at	city_id → ol_service_areas	–
`ol_discover_chips`	`id text`	label NN; emoji; color; sort_order	–	–
`ol_coupons`	`code text`	label NN; kind NN CHECK `flat/pct`; value numeric(10,2) NN CHECK >0; min_order numeric(10,2) NN 0; description NN ''; active NN true; city_id NN; is_demo; created_at. **0026 adds** store_id text	city_id → ol_service_areas; 0026 store_id → ol_stores(id) CASCADE	–
`ol_events`	`id text`	title NN; kind NN CHECK `movies/events/activities`; subtitle; venue; price numeric(10,2); image; tag; slots text[] NN '{}'; city_id NN; is_demo; sort_order; created_at. **Not an event log** — this is the "Book It" listing table	city_id → ol_service_areas	–
`ol_reels`	`id text`	product_id; brand_id; video NN; caption; creator; likes int NN 0; tag; city_id NN; is_demo; sort_order	product_id → ol_products CASCADE; brand_id → ol_brands CASCADE; city_id	–
`ol_themed_stores`	`id text`	data jsonb NN; city_id; is_demo; sort_order; created_at	city_id	–
`ol_media_assets`	`key text`	bucket NN 'onecity-catalog'; path NN; content_type NN; bytes int; created_at	–	No reader or writer found in `frontend/` or `partner/` (UNKNOWN whether the legacy `backend/` Python writes it)

1.2 Orders, lines, refunds — 0021, 0022, 0024, 0025, 0026

`ol_orders` — PK `id uuid default gen_random_uuid()`.

Column	Type / default / constraint	Added in
customer_hash	text NN	0021
user_id	uuid → auth.users SET NULL	0021
items	jsonb NN (read cache of lines)	0021
subtotal	numeric(10,2) NN	0021
discount	numeric(10,2) NN 0	0021
delivery_fee	numeric(10,2) NN 0	0021
total	numeric(10,2) NN	0021
coupon_code	text	0021

Column	Type / default / constraint	Added in
payment_method	text NN 'cod'; CHECK `cod,upi,card,netbanking` → widened by 0025 to `+wallet,paylater,emi,cardless_emi` (constraint `ol_orders_payment_method_check`)	0021/0025
payment_status	text NN 'not_charged'; CHECK `not_charged,pending,paid,failed,refunded`	0021
status	text NN 'placed'; CHECK `placed,accepted,packed,out,delivered,cancelled`	0021
rider_name, rider_phone, eta_minutes	text, text, int	0021
delivery_address	jsonb	0021
idempotency_key	text	0021
pos_order_id	uuid → orders(id) SET NULL	0021
city_id	text NN 'latur' → ol_service_areas	0021
created_at, updated_at	timestamptz NN now(); trigger `ol_orders_updated_at` → `set_updated_at()`	0021
organization_id	uuid → organizations SET NULL	0022
restaurant_deadline	timestamptz	0022
prep_minutes	int CHECK 5..120	0022
gst	numeric(10,2) NN 0	0022
cancel_reason	text	0022
cancelled_by	uuid → auth.users SET NULL (**never written by any code in this repo**)	0024
razorpay_order_id, razorpay_payment_id, paid_at, payment_error	text, text, timestamptz, text CHECK ≤300 chars	0025
checkout_id	uuid → ol_checkouts SET NULL	0026
store_id	text → ol_stores SET NULL	0026

Indexes: `ol_orders_idempotency` UNIQUE (customer_hash, idempotency_key) WHERE idempotency_key NOT NULL; `ol_orders_customer (customer_hash, created_at desc)`; `ol_orders_user (user_id, created_at desc)` WHERE user_id NOT NULL; `ol_orders_org_status (organization_id, status, created_at desc)` WHERE organization_id NOT NULL; 0025 `ol_orders_razorpay_order` UNIQUE, `ol_orders_razorpay_payment` UNIQUE (both partial); 0026 `ol_orders_checkout`, `ol_orders_store (store_id, created_at desc)`.

`ol_order_items` (0024) — PK `id uuid`. `order_id uuid NN → ol_orders CASCADE`; `product_id text → ol_products(id) SET NULL`; `menu_item_id uuid → menu_items(id) SET NULL`; name NN; weight NN ''; image NN ''; price numeric(10,2) NN ≥0; mrp NN ≥0; qty int NN 1..99; discount numeric(10,2) NN 0; `line_total` **GENERATED ALWAYS AS (price*qty - discount) STORED**; refunded_qty int NN 0 CHECK 0..qty; created_at. Indexes `ol_order_items_order`, `ol_order_items_product` (partial).

`ol_refunds` (0024) — PK `id uuid`. order_id NN → ol_orders CASCADE; order_item_id → ol_order_items SET NULL; amount numeric(10,2) NN >0; reason text NN CHECK 3..500 chars; method NN 'original' CHECK `original,wallet,manual`; status NN 'requested' CHECK `requested,approved,rejected,processed`; reference text; requested_by, processed_by → auth.users SET NULL; created_at; processed_at. Indexes `ol_refunds_order`; 0025 `ol_refunds_reference` UNIQUE WHERE reference NOT NULL.

`ol_checkouts` (0026, DRAFT) — PK `id uuid`. customer_hash NN; user_id → auth.users SET NULL; kind NN CHECK `food,shopping`; item_total numeric(10,2) NN ≥0; discount NN 0 ≥0; delivery_fee NN 0 ≥0; total NN ≥0; coupon_code; payment_method NN 'cod' (CHECK 8 values); payment_status NN 'not_charged' (CHECK 5 values); razorpay_order_id; razorpay_payment_id; paid_at; payment_error (≤300); idempotency_key; city_id NN 'latur'; created_at; updated_at (trigger `ol_checkouts_updated_at`). Indexes: UNIQUE (customer_hash, idempotency_key) partial; UNIQUE razorpay_order_id partial; UNIQUE razorpay_payment_id partial; `ol_checkouts_customer`; `ol_checkouts_user`. Trigger `ol_checkouts_payment_to_orders` (AFTER UPDATE OF payment_method, payment_status, paid_at, payment_error → function `ol_checkout_payment_to_orders()` copies those four fields to every `ol_orders` row with that `checkout_id`).

`ol_payment_events` (0025; `checkout_id` added by 0026) — PK `event_id text` (Razorpay's event id). event NN; order_id → ol_orders SET NULL; razorpay_payment_id; created_at; checkout_id → ol_checkouts SET NULL. Indexes on order_id and checkout_id.

1.3 Restaurant (VINII Partner) tables — 0022

`ol_merchant_channels` — PK `organization_id uuid → organizations CASCADE`. store_id text → ol_stores SET NULL; accepting bool NN false; back_at; accept_window_seconds int NN 60 CHECK 15..600; default_prep_minutes int NN 20 CHECK 5..120; missed_in_a_row int NN 0 ≥0; auto_pause_after int NN 3 CHECK 1..20; address; lat; lng; city_id NN 'latur'; updated_by → auth.users SET NULL; updated_at; trigger `ol_merchant_channels_updated_at`.

`ol_partner_devices` — PK `id uuid`. organization_id NN → organizations CASCADE; user_id NN → auth.users CASCADE; `expo_push_token text NN UNIQUE`; platform NN CHECK `ios,android,web`; last_seen_at; created_at. Index `ol_partner_devices_org`.

1.4 Help / bookings / privacy

`ol_booking_enquiries` (0021, +0024) — PK uuid. event_id NN → ol_events; name NN CHECK 2..80 chars; phone NN CHECK `^[6-9][0-9]{9}$`; date NN; slot NN; guests int NN 1..8; status NN 'demo_enquiry_saved'; customer_hash; user_id → auth.users SET NULL; created_at; resolved_by → auth.users SET NULL; resolved_at.

`ol_support_requests` (0021, +0024) — PK uuid. topic NN CHECK `order,payment,profile,other`; message NN CHECK 10..2000; reference NN ''; status NN 'saved_for_review'; customer_hash; user_id; created_at; resolved_by; resolved_at. **Overloaded:** also stores "launch here" demand (`reference = 'launch:<place id>'`) and, before 0027, privacy requests (`reference = 'privacy:<ref>'`).

`ol_consents` (0027, DRAFT) — PK `id bigint generated always as identity`. user_id uuid NN (**deliberately no FK**); purpose NN CHECK `marketing,personalisation,location_suggestions,notice_acknowledged,age_18_plus`; granted bool NN; notice_version text NN 1..40; source NN CHECK `sign_in,privacy_screen,consent_manager,support`; given_at; created_at. Index `ol_consents_user (user_id,purpose,created_at desc)`. Trigger `ol_consents_append_only` (BEFORE UPDATE OR DELETE; raises `OL_BAD_REQUEST` unless DELETE with `ol.privacy_purge = 'on'`).

`ol_privacy_requests` (0027, DRAFT) — PK uuid. user_id uuid (no FK); kind NN CHECK `export,erasure,correction,nominee,grievance`; status NN 'received' CHECK `received,in_progress,completed,partially_completed,rejected,withdrawn`; reference text NN UNIQUE CHECK `^[A-Z]{2}-[0-9]{6}-[A-Z0-9]{6}$`; details jsonb NN '{}'; due_at; resolved_at; created_at; updated_at (trigger). Indexes `ol_privacy_requests_user`, `ol_privacy_requests_open` (partial on open statuses).

1.5 Delivery / dispatch tables — 0029 (DRAFT; tables are not `ol_`-prefixed)

Table	PK	Columns	FKs / uniques / checks
`riders`	id uuid	user_id uuid NN UNIQUE; full_name NN; phone; type NN 'store' CHECK `store,solo`; status NN 'draft' CHECK `draft,submitted,verification_pending,approved,rejected,documents_expired,suspended`; organization_id; vehicle CHECK `bike,scooter,cycle,car`; acceptance NN 'manual' CHECK `auto,manual`; cash_limit ≥0; created_at; updated_at	user_id → auth.users CASCADE; organization_id → organizations SET NULL; trigger `riders_updated_at`
`rider_store_links`	(rider_id, store_id)	priority int NN 1 ≥1; status NN 'active' CHECK `active,paused`	rider_id → riders CASCADE; store_id → ol_stores CASCADE
`rider_availability`	rider_id	online bool NN false; lat CHECK ±90; lng CHECK ±180; accuracy_m; battery CHECK 0..100; last_seen_at	rider_id → riders CASCADE; index `rider_availability_online (last_seen_at) WHERE online`
`push_tokens`	id uuid	rider_id NN; `expo_push_token NN UNIQUE`; platform NN CHECK `ios,android`; last_ok_at; created_at	rider_id → riders CASCADE; index `push_tokens_rider`
`org_dispatch_settings`	organization_id	dispatch_trigger NN 'placed' CHECK `placed,accepted`; offer_seconds NN 30 (10..120); max_rounds NN 3 (1..20); search_radius_km numeric(4,1) NN 8 (1..50); updated_at	organization_id → organizations CASCADE; trigger
`delivery_jobs`	id uuid	`ol_order_id uuid NN UNIQUE`; pos_order_id uuid (**no FK**); organization_id; store_id; city_id NN 'latur'; status NN 'created' CHECK `created,offered,assigned,at_pickup,picked_up,at_drop,delivered,exception,cancelled`; version int NN 1; `idempotency_key text NN UNIQUE`; rider_id; payment_mode NN CHECK `cod,online`; cash_to_collect numeric(10,2) NN 0; items_count int NN 1; temperature NN 'ambient' CHECK `hot,cold,ambient`; vehicle NN 'any' CHECK `any,bike,scooter,cycle,car`; prep_minutes 0..240; promised_by; order_no text; kitchen_no text; pickup_name NN; pickup_address; pickup_lat/lng NN (±90/±180); pickup_phone; drop_area; drop_address; drop_lat/lng NN; drop_notes; customer_first_name; customer_phone; **otp_hash text NN**; otp_attempts int NN 0; dispatch_trigger NN 'placed'; released_at; rounds_base int NN 0; escalated_at; cancel_reason; created_at; updated_at	ol_order_id → ol_orders **ON DELETE RESTRICT**; organization_id → organizations SET NULL; store_id → ol_stores SET NULL; city_id → ol_service_areas; rider_id → riders SET NULL. Indexes `delivery_jobs_rider`, `delivery_jobs_open`, `delivery_jobs_org`; trigger

Table	PK	Columns	FKs / uniques / checks
`delivery_assignments`	id uuid	job_id NN; rider_id NN; round int NN ≥1; offered_at; expires_at NN; outcome CHECK `accepted,rejected,timed_out,withdrawn`; reason; answered_at	job_id → delivery_jobs CASCADE; rider_id → riders CASCADE. UNIQUE partial: `delivery_one_live_offer_per_job (job_id) WHERE outcome IS NULL`, `delivery_one_live_offer_per_rider (rider_id) WHERE outcome IS NULL`, `delivery_one_accepted_per_job (job_id) WHERE outcome='accepted'`, `delivery_one_offer_per_round (job_id, round)`; index `delivery_assignments_rider`
`delivery_events`	`id` bigserial	job_id NN; kind text NN; v int NN 1; meta jsonb NN '{}'; created_at	job_id → delivery_jobs CASCADE; index `delivery_events_job (job_id,id)`; sequence `delivery_events_id_seq`

View `delivery_orders_needing_rider` (0029): `ol_orders` rows with status not in (cancelled,delivered), created between 24 h and 1 min ago, `(payment_method='cod' OR payment_status='paid')`, and no `delivery_jobs` row. Grants: `select` to service_role only.

2. Views, materialized views, sequences

Object	Kind	Source
`ol_consent_current`	view, `security_invoker = true`: newest `ol_consents` row per (user_id, purpose)	0027 DRAFT
`delivery_orders_needing_rider`	view	0029 DRAFT
`delivery_events_id_seq`	sequence (bigserial)	0029
`ol_consents.id`	identity sequence	0027
Materialized views	**none**	–

3. Functions / RPCs (all `SECURITY INVOKER`, `set search_path = public, pg_temp`; EXECUTE revoked from `public, anon, authenticated`, granted to `service_role` only)

Function	Migration	Called from	Purpose
`ol_create_restaurant_order(p jsonb) → jsonb`	0023	`frontend/server/restaurant.ts createTestOrder` **only**	Creates `orders` + `order_items` + `order_events('placed')` in POS tables and `ol_orders` row, atomically; prices from `menu_items`; GST 5%; idempotent on (customer_hash, idempotency_key)
`ol_partner_act(p_order, p_action, p_actor, p_actor_name, p_expected, p_prep, p_reason) → jsonb`	0023	`restaurant.ts act` (partner accept/reject/ready/out/delivered) and `ol_expire_unanswered` (timeout)	State machine across `orders`, `order_items`, `kot_tickets`, `payments`, `order_events`, `ol_orders`, `ol_merchant_channels`; optimistic lock on `orders.updated_at`
`ol_expire_unanswered(p_org uuid default null) → int`	0023	`restaurant.ts listOrders` (every partner list read)	Times out unanswered orders
`ol_process_refund(p_order, p_item, p_amount, p_reason, p_actor, p_method) → jsonb`	0024	`store.ts refundOrder` — **which no route or code calls** (INFERRED: support tooling not built)	Writes `ol_refunds` (status 'processed'), bumps `ol_order_items.refunded_qty`, flips `ol_orders.payment_status` to `refunded`
`ol_checkout_payment_to_orders()`	0026	trigger	Copies payment state checkout → orders
`ol_consents_append_only()`	0027	trigger	Evidence is immutable
`dispatch_km(a_lat,a_lng,b_lat,b_lng) → double precision`	0029	other dispatch functions	Haversine km (IMMUTABLE SQL)
`dispatch_create_job(p jsonb) → jsonb`	0029	`dispatch.ts createJob`	Creates `delivery_jobs` + `delivery_events('delivery.created')`; idempotent per `ol_order_id`
`dispatch_release(p_job, p_prep, p_expected, p_request_id) → jsonb`	0029	`dispatch.ts releaseJob`	Starts offering
`dispatch_offer_next(p_job, p_request_id) → jsonb`	0029	`dispatch.ts advanceJob`	Lapses overdue offer, picks next rider, or escalates

Function	Migration	Called from	Purpose
`dispatch_answer(p_assignment, p_rider, p_accept, p_reason, p_request_id) → jsonb`	0029	`rider.ts` accept/decline, `dispatch.ts` auto-accept	Rider accepts/declines
`dispatch_cancel(p_job, p_reason, p_expected, p_request_id) → jsonb`	0029	`dispatch.ts` `cancelJob`	Cancels job; withdraws live offer
`dispatch_due_jobs(p_limit int) → setof uuid` (STABLE SQL)	0029	`dispatch.ts` `sweep`, `advanceDue`	Jobs needing the next step

OS/POS functions used (not owned): `next_org_seq(org uuid, counter_kind text) → bigint` (SECURITY DEFINER; upserts `org_counters`), `set_updated_at() → trigger`. Source `db/local/0000_os_replica.sql`.

4. Triggers

Trigger	Table	Function
`ol_orders_updated_at`	ol_orders	`set_updated_at`
`ol_merchant_channels_updated_at`	ol_merchant_channels	`set_updated_at`
`ol_checkouts_updated_at`	ol_checkouts	`set_updated_at`
`ol_checkouts_payment_to_orders`	ol_checkouts	`ol_checkout_payment_to_orders`
`ol_stores_updated_at`	ol_stores	`set_updated_at`
`ol_consents_append_only`	ol_consents	`ol_consents_append_only`
`ol_privacy_requests_updated_at`	ol_privacy_requests	`set_updated_at`
`riders_updated_at`, `org_dispatch_settings_updated_at`, `delivery_jobs_updated_at`	riders, org_dispatch_settings, delivery_jobs	`set_updated_at`

No trigger writes to an outbox/event table. `ol_order_events` (named in docs as the future outbox) **does not exist** in any migration.

5. Storage buckets and realtime

Item	Detail
Bucket `onecity-catalog`	Created in 0021: `public = true`, `file_size_limit = 41943040` (40 MiB), mime `image/jpeg, image/png, image/webp, video/mp4, video/webm`. Written via signed upload URL from `admin.ts uploadUrl` (path `stores/YYYY/MM/<hex>-<slug>.<ext>`, 5 MB check in code). **No storage policies are defined by this repo** (UNKNOWN whether Supabase defaults apply)
Bucket `item-images`	POS bucket (public) per `CENTRAL_DB_MAP.md`; not touched by this repo
Realtime	`supabase/config.toml [realtime] enabled = true` (local). **No `ol_*` table is added to a publication in any migration; no client code subscribes** (`grep` for `.channel(` / `postgres_changes` finds nothing). Phones poll. `docs/PLATFORM_FOUNDATIONS.md` D5 says realtime *listening* is intended later
Supabase Auth	Used for sign-in (email+password, phone OTP, Google OAuth PKCE); see `06_RLS_SECURITY.md`

6. OS / POS tables this repo reads or writes (owned elsewhere)

Columns below are from `db/local/0000_os_replica.sql` (generated from production's catalog) — treat as a snapshot. Policies on these tables are **UNKNOWN** from this repo (see 06).

Table	PK	Columns	Notable constraints
`organizations`	id uuid	name, slug, type(org_type), status(org_status), legal_name, gstin, contact_email, contact_phone, settings, created_by, created_at, updated_at, theme, source_lead_id, onboarding_stage, onboarding_owner, lifecycle_stage	slug format, gstin format, name not blank, theme shape

Table	PK	Columns	Notable constraints
`locations`	id uuid	organization_id, name, kind, code, address, city, franchisee_name, is_primary, created_by, created_at, updated_at (per `CENTRAL_DB_MAP.md` ; **not in the replica**)	–
`roles`	id	slug UNIQUE, name, description, is_system, created_at	slug format
`org_users`	id	organization_id, user_id, role_id, email, full_name, status(member_status), created_at, updated_at	UNIQUE (organization_id, user_id)
`org_counters`	(organization_id, kind)	value int8	–
`menu_categories`	id	organization_id, name, sort_order, created_at	–
`menu_items`	id	organization_id, category_id, name, sku, price numeric(10,2) ≥0, image_url, is_available, sort_order, description, food_type(veg/non_veg/egg), course(eat/drink), is_recommended, prep_minutes, station	–
`menu_item_variants`	id	organization_id, menu_item_id, name, price, is_default, sort_order (per `CENTRAL_DB_MAP.md` ; not in the replica)	–
`item_channel_status`	(menu_item_id, channel)	organization_id, is_available, off_until, updated_at	One Local writes `channel='one_local'`
`channel_integrations`	id	organization_id, channel CHECK swiggy/zomato/ondc/website/whatsapp/qr, display_name, is_connected, is_accepting, next_open_at, auto_accept, commission_pct, config	UNIQUE (organization_id, channel). **One Local does not use it** (it has its own `ol_merchant_channels`)
`orders`	id	organization_id, order_no int8, display_no, channel(dine_in/delivery/pickup/online/swiggy/zomato/other), status(new/in_billing/sent_to_kitchen/ready/awaiting_payment/paid/delivered/cancelled), table_id, customer_name, customer_phone, captain_name, subtotal, gst_pct, gst_amount, total, payment_method(cash/upi/card/split), created_by, created_at, updated_at, session_id, placed_by(pos/captain/guest/integration), guest_count, discount, service_charge, round_off, paid_at, notes, external_ref	UNIQUE (organization_id, order_no)
`order_items`	id	order_id, menu_item_id, name, qty>0, unit_price≥0, add_ons jsonb, notes, created_at, variant_name, status(pending/preparing/ready/delivered/cancelled), kot_ticket_id, course, station	–
`order_events`	id	organization_id, order_id, kind, message, actor, meta jsonb, created_at	–
`kot_tickets`	id	organization_id, order_id, kot_no int8, station, status(new/preparing/ready/delivered), priority(normal/urgent)	UNIQUE (organization_id, kot_no)
`payments`	id	organization_id, order_id, method(cash/upi/card/wallet/due/other), amount>0, reference, created_by, created_at	–
`platform_admins`	user_id	email, created_at, role, full_name (per `CENTRAL_DB_MAP.md`)	read by `admin.ts`
`auth.users`	id	Supabase Auth; read through the Admin API (`auth.admin.getUserById/listUsers/updateUserById/deleteUser`)	–

Tables named in `CENTRAL_DB_MAP.md` and relevant to the comparison but **not used** by this repo: `dining_tables` , `table_sessions` , `dining_areas` , `kitchen_stations` , `service_requests` , `customers` , `invoices` , `notifications` .

02 — ENTITY MAP and IDENTITY (One Local / OL-ui)

Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**; **DRAFT** = migration not applied to production per `docs/JOURNAL.md`. "Server" below always means `frontend/server/*` running behind `frontend/app/api/*` with the Supabase **service role** key (see `06_RLS_SECURITY.md`). The apps never talk to the database directly.

1. Entity inventory

ENTITY | TABLE | CREATED BY | UPDATED BY | READ BY | PRIMARY KEY | IMPORTANT FOREIGN KEYS | SOURCE OF TRUTH | NOTES

ENTITY	TABLE	CREATED BY	UPDATED BY	READ BY	PRIMARY KEY	IMPORTANT FOREIGN KEYS	SOURCE OF TRUTH	NOTES
User (login)	`auth.users` (Supabase)	Supabase Auth: customer app `signUp`, `signInWithOtp` (phone), Google OAuth (`frontend/src/auth.tsx`); partner app email+password sign-in only (`partner/src/auth.tsx`); rider login = phone OTP (docs, rider app not in repo)	Supabase Auth; `admin.ts blockShopper` (`ban_duration`); `privacy.ts endLogin` (global logout / `deleteUser` only if `PRIVACY_HARD_DELETE=on`)	Server: `signedInUserId` → `GET {SUPABASE_URL}/auth/v1/user`; `admin.ts authUsers` → `auth.admin.listUsers`; `dispatch.ts` → `auth.admin.getUserById` (name, phone)	`id uuid`	referenced by `ol_orders.user_id`, `ol_checkouts.user_id`, `org_users.user_id`, `riders.user_id`, `platform_admins.user_id`, `ol_partner_devices.user_id`	Supabase Auth	One login shared by customers, restaurant staff, riders, platform admins (`docs/PLATFORM_FOUNDATIONS.md` D1). No `shoppers` / `profiles` table exists (docs name `shoppers` as a future membership table; **no migration creates it**)
Organization (restaurant / merchant business)	`organizations` (OS-owned)	OS/Master (outside this repo)	OS/Master	`admin.ts` (`readOrganizations`), `partner.ts` (join), `marketplace.ts`	`id uuid`	referenced by `ol_stores.organization_id`, `ol_orders.organization_id`, `ol_merchant_channels.organization_id` (PK), `riders.organization_id`, `org_dispatch_settings`, `delivery_jobs`	OS (Master)	One Local never writes it
Location / outlet	`locations` (OS-owned; **not in the local replica**)	OS/Master	OS/Master	`marketplace.ts readLocations` (id, organization_id, name, city, address, code)	`id uuid`	`ol_stores.location_id` (0028 DRAFT)	OS	UNKNOWN whether `locations` has rows (`CENTRAL_DB_MAP.md` snapshot: 0 rows). **Distinct from** `ol_service_areas.city_id` and `ol_merchant_channels.address/lat/lng`
City / service area	`ol_service_areas`	seed in 0021 (`latur`); `admin.ts` has no create route	– (rows edited by SQL)	`coverage.ts liveAreas` (60 s cache), `admin.ts requireCity`; `catalog.ts` also has a hardcoded `SERVICE_AREAS` array	`city_id text`	referenced by every `city_id` column	`ol_service_areas` (DB), **duplicated** by `ACTIVE_CITIES` / `SERVICE_AREAS` constants in `catalog.ts`	See 10
Customer (shopper)	**no table.** A customer is `auth.users` + the hash columns `customer_hash` on `ol_orders / ol_checkouts / ol_booking_enquiries / ol_support_requests`	`store.ts` (hash computed in `auth.ts caller`)	privacy erasure rewrites hash to `erased:<uuid>`	`store.ts`, `privacy.ts`	n/a	`user_id → auth.users`	Supabase Auth for identity; hash for ownership	Profile, addresses, wishlist, preferences live only in the phone's AsyncStorage (`frontend/src/customer.tsx`)
`customers` (OS CRM)	`customers` (OS)	OS	OS	not used by this repo	–	–	OS	Name collision: the OS `customers` table is a **sales CRM** (business customers), not marketplace shoppers (`PLATFO`

ENTITY	TABLE	CREATED BY	UPDATED BY	READ BY	PRIMARY KEY	IMPORTANT FOREIGN KEYS	SOURCE OF TRUTH	NOTES
								RM_FOUNDATIONS.md D2 explains)
Staff	org_users (OS) + roles (OS)	OS/Master invites (outside repo)	OS	partner.ts requirePartnerAuth (reads user_id,email,full_name,role:roles(slug),organization:organizations(id,name) where status='active', earliest membership) ; admin.ts isMerchantStaff	id uuid	organization_id, user_id, role_id	OS	Roles checked in code: org_owner, org_admin, manager, biller, location_manager, operations_manager, kitchen (partner.ts CAN)
Platform admin	platform_admins (OS)	OS	OS	admin.ts platformAdmin (30 s cache)	user_id	–	OS	Write roles in code: master_admin, onboarding_admin, operations ; dev roles list in marketplace.ts DEV_ROLES
Product (marketplace)	ol_products (demo catalogue) **and** virtual products derived at read time from menu_items + menu_item_variants of live listings (marketplace.ts listingCatalog)	demo: frontend/scripts/seed-catalog.mjs ; real: POS item master (outside repo)	demo: SQL; real: POS	catalog.ts getCatalog (30 s cache), pricing.ts quote	demo: id text (p1 …); virtual: "<listing id>_<menu_item uuid>[_<variant uuid>]"	demo: category_id, menu_item_id	Real: POS menu_items . Demo: ol_products (0 rows after clean-up per journal, DOC-CLAIM)	Two parallel product sources — see 10
Category	ol_categories (demo) and virtual im-<department>-<slug> from menu_categories	as above	–	catalog.ts	id text	–	POS menu_categories for real	
Variant	menu_item_variants (OS/POS, not in replica)	POS	POS	marketplace.ts readItemMaster (optional)	id uuid	menu_item_id	POS	Each variant becomes its own One Local product id
SKU	menu_items.sku (POS column)	POS	POS	**not read** by this repo	–	–	–	No SKU concept in ol_*
Inventory / stock	**none**	–	–	–	–	–	–	Availability only: ol_products.is_available, menu_items.is_available, item_channel_status (channel='one_local', off_until). No quantity on hand, no decrement on order
Store / listing	ol_stores	Master via POST /api/admin/listings (admin.ts createListing , status pending); demo seed	admin.ts updateListing / setListingStatus	catalog.ts, dispatch.ts createJobForOrder (pickup point), admin.ts	id text (demo s1 …; Master-created org-<slug>[-<outlet>])	organization_id, location_id (0028), city_id	ol_stores (listing), POS (items)	
Merchant channel (accepting switch, pickup address)	ol_merchant_channels	admin.ts saveChannel (insert, accepting=false)	restaurant.ts setOutlet ; ol_partner_act (missed counter, auto-pause); admin.ts saveChannel	restaurant.ts getOutlet , dispatch.ts , marketplace.ts readChannels	organization_id uuid	organization_id, store_id, city_id	ol_merchant_channels	One row per organization (not per outlet). The POS has channel_integrations for the same idea — see 10
Order (custom	ol_orders	store.ts createCheckout (+ ol_create_restaurant_o	ol_partner_act (status, prep, eta, payment_status,	store.ts listOrders/getOrder , restaurant.ts , di	id uuid (generated by the	checkout_id, store_id, organization_id, pos_o	ol_orders for customer side; or	See 03_ORDER_LIFECYCLE.md for the critical gap: orders

ENTITY	TABLE	CREATED BY	UPDATED BY	READ BY	PRIMARY KEY	IMPORTANT FOREIGN KEYS	SOURCE OF TRUTH	NOTES
er side)		`rder` for the Partner test order)	cancel_reason); trigger from `ol_checkouts` (payment fields); `ol_process_refund` and `store.ts recordGatewayRefund` (`payment_status`); privacy erasure	`spatch.ts`, `privacy.ts`	server with `crypto.randomUUID()` for checkout orders; DB default for the SQL function)	`rder_id`, `user_id`, `city_id`	`ders` (POS) for kitchen side — only linked when created by `ol_create_restaurant_order`	created by customer checkout have **no** `organization_id` / `pos_order_id`
Checkout (one payment, many store orders)	`ol_checkouts` (**DRAFT** 0026)	`store.ts createCheckout`	`store.ts updatePayment` (payment fields), `recordGatewayRefund`	`store.ts`, `payments.ts`	`id uuid`	`user_id`	`ol_checkouts`	Absent in production per journal → server falls back to per-order mode, COD only
Order line	`ol_order_items` (0024) and jsonb `ol_orders.items`	`store.ts createCheckout` (best effort, after orders commit)	`ol_process_refund` (`refunded_qty`)	`ol_process_refund` only; `privacy` export does not select it	`id uuid`	`order_id`, `product_id → ol_products`, `menu_item_id → menu_items`	**Declared** truth = `ol_order_items`; **actually read** = jsonb `items`	`store.ts` never fills `menu_item_id`; `product_id` is set to the catalogue id which, for marketplace products, is not in `ol_products` (INFERRED FK violation, see 10)
Kitchen order	`orders` (POS) + `order_items` (POS)	`ol_create_restaurant_order`	`ol_partner_act`	`restaurant.ts` (joined via `ol_orders.pos_order_id`)	`id uuid`; `order_no int8` per org; `display_no text`	`organization_id`, `table_id`	POS	
KOT / kitchen ticket	`kot_tickets` (POS)	`ol_partner_act` on **accept** (`next_org_seq(org,'kot')`, station `'main'`, status `new`)	`ol_partner_act` ready (→ `ready`) and out (→ `delivered`)	`order_items.kot_ticket_id`	`id uuid`; `kot_no int8` unique per org	`order_id`, `organization_id`	POS	One KOT per accepted order; items all station `'main'` unless item has another (`order_items.station` copied from menu item but KOT station fixed `'main'`)
Payment (customer side)	**no payments table.** State is on `ol_checkouts` (or `ol_orders` before 0026): `payment_method`, `payment_status`, `razorpay_order_id`, `razorpay_payment_id`, `paid_at`, `payment_error`	`store.ts createCheckout`; `payments.ts startPayment`	`payments.ts markPaid / handleWebhook / switchToCash`; `store.ts updatePayment`	`store.ts findPayment`	checkout `id uuid`	–	Razorpay (money), `ol_checkouts` (our record)	
Payment (POS side)	`payments` (POS)	`ol_partner_act` on `delivered` (method `'cash'`, amount `orders.total`, reference `'onelocal:<ol_orders.id>'`)	–	–	`id uuid`	`order_id`, `organization_id`	POS	Only for POS-linked (restaurant) orders
Payment webhook log	`ol_payment_events`	`payments.ts remember`	–	`payments.ts alreadyHandled`	`event_id text`	`order_id`, `checkout_id`	`ol_payment_events`	
Refund	`ol_refunds`	`ol_process_refund` (function, never invoked by a route); `store.ts recordGatewayRefund` (Razorpay `refund.processed` webhook)	same	`privacy.ts` export	`id uuid`	`order_id`, `order_item_id`	`ol_refunds`	Unique on `reference` (Razorpay refund id)
Invoice	**none for orders.** `invoices` (OS) is a SaaS-	–	–	not used	–	–	–	No GST invoice document or number is generated for orders. GST amounts only stored as `ol`

ENTITY	TABLE	CREATED BY	UPDATED BY	READ BY	PRIMARY KEY	IMPORTANT FOREIGN KEYS	SOURCE OF TRUTH	NOTES
	subscription invoice (`subscription_id`, `period_*`)							`_orders.gst` and shown in the bill
Dining table	`dining_tables`, `table_sessions` (OS/POS)	–	–	**not used**	–	–	–	`orders.table_id` stays NULL for One Local orders
Delivery job	`delivery_jobs` (**DRAFT** 0029)	`dispatch.ts createJob` via `dispatch_create_job`	`dispatch_release/offer_next/answer/cancel`	`dispatch.ts`, `rider.ts`	`id uuid`	`ol_order_id` UNIQUE → `ol_orders` (RESTRICT)	`delivery_jobs`	
Offer / assignment	`delivery_assignments` (DRAFT)	`dispatch_offer_next`	`dispatch_answer`, `dispatch_cancel`, timeout in `dispatch_offer_next`	`rider.ts currentOffer`	`id uuid`	`job_id`, `rider_id`	same	
Delivery event (outbox)	`delivery_events` (DRAFT)	all dispatch functions	– (append only)	`dispatch.ts jobView` timeline	`id bigserial`	`job_id`	same	**Only outbox in the repo.** No consumer process exists in this repo
Rider	`riders` (DRAFT)	**no creation path in this repo** — approved by hand in the database (journal)	`riders.status` by hand	`rider.ts requireRider`, `dispatch_offer_next`	`id uuid` (+ `user_id` UNIQUE)	`user_id`, `organization_id`	`riders`	Rider app code is not in this repo
Rider presence	`rider_availability` (DRAFT)	`rider.ts setAvailability` (upsert)	`rider.ts heartbeat`	`dispatch_offer_next`, `dispatch.ts jobView`	`rider_id`	`rider_id`	same	Position from the rider's phone every ~30 s
Address	**no table.** Delivery address stored as jsonb `ol_orders.delivery_address` `{label,text,lat,lng,notes}`; saved addresses only in AsyncStorage (`onecity.customer.v1`)	`store.ts readAddress`	privacy erasure sets to NULL	`restaurant.ts`, `dispatch.ts`	–	–	Phone (saved) / `ol_orders.delivery_address` (per order)	Two shapes: server shape `{label,text,lat,lng,notes}` vs app `Address {id,label,name,phone,line,area,pin,kind,lat,lng,building,street,locality,formatted,instructions}`
Notification (push)	**no table.** Tokens: `ol_partner_devices` (restaurant), `push_tokens` (rider, DRAFT). Customers: none server-side	`restaurant.ts registerDevice`; `rider.ts registerPushToken`	–	`push.ts`	–	–	–	OS has `notifications` (team inbox) — unused here. Delivery to Expo Push API
Promotion / coupon	`ol_coupons`	seed / SQL	–	`catalog.ts applyCoupon`, `pricing.ts`	`code text`	`city_id`, `store_id` (0026)	`ol_coupons`	
Discount	no table. `ol_orders.discount`, `ol_checkouts.discount`, `ol_order_items.discount` (never written non-zero by code)	`pricing.ts quote` → `store.ts`	–	–	–	–	computed per request	See 03
Banner / chips / reels / themed stores / events listing	`ol_banners`, `ol_discover_chips`, `ol_reels`, `ol_themed_stores`, `ol_events`	seed	–	`catalog.ts`	`id text`	–	`ol_*`	Content tables
Booking enquiry	`ol_booking_enquiries`	`store.ts createEnquiry`	privacy erasure deletes	`privacy.ts` export	`id uuid`	`event_id → ol_events`	`ol_booking_enquiries`	Status text `'demo_enquiry_saved'`; no processing flow

ENTITY	TABLE	CREATED BY	UPDATED BY	READ BY	PRIMARY KEY	IMPORTANT FOREIGN KEYS	SOURCE OF TRUTH	NOTES
Support / help / launch-demand request	`ol_support_requests`	`store.ts createSupportRequest`, `createLaunchRequest`; `privacy.ts` fallback	privacy erasure	`admin.ts demand`, `privacy.ts`	`id uuid`	–	same	One table, three meanings (see 10)
Consent	`ol_consents` (DRAFT 0027)	`privacy.ts recordConsents`	– (append only)	`privacy.ts currentConsents`	`id bigint`	none (`user_id` uuid, no FK)	`ol_consents`	
Privacy request	`ol_privacy_requests` (DRAFT)	`privacy.ts saveRequest`	`privacy.ts updateRequest`	export	`id uuid`	none	same	
Wallet / Coins	**none.** `frontend/app/wallet.tsx` keeps a demo balance in AsyncStorage key `onecity-demo-wallet`; `features().wallet` / `coins` default `false`	–	–	–	–	–	–	`ol_refunds.method='wallet'` is allowed but nothing credits a wallet
Media asset	`ol_media_assets` (unused) + Storage `onecity-catalog`	`admin.ts uploadUrl` (Storage only)	–	`/api/media/<key>` redirects via `frontend/server/data/media.json`	`key text`	–	Storage + `media.json`	

2. IDENTITY

ID	Where generated	Type	DB- or app-generated	Uniqueness scope	Reused across systems?
user ID	Supabase Auth on sign-up	uuid	Supabase	global (`auth.users`)	**Yes** — same login for customer app, Partner app, rider, Master (D1). Written to `ol_orders.user_id`, `org_users.user_id`, `riders.user_id`, `platform_admins.user_id`
organization ID	OS (outside repo)	uuid	DB default in OS	global	**Yes** — `ol_orders.organization_id`, `ol_stores.organization_id`, POS `orders.organization_id`
location ID	OS `locations.id`	uuid	OS	global	Only as `ol_stores.location_id` (0028 DRAFT). Customer "location" is a different thing: `city_id text` (`'latur'`) plus the delivery address lat/lng
city ID	`ol_service_areas.city_id`	text slug `latur`	manual	global	Every `city_id` column; HTTP `?city=` / `x-city` header (`catalog.ts resolveCity`)
device ID	`frontend/src/api.ts randomId(24)` → 48 hex chars, saved at AsyncStorage key `onecity.customer-key.v1`, sent as header `x-customer-key`. Server regex: `^[A-Za-z0-9_-]{24,128}$`	string	app-generated (client)	per phone install; new key after "Clear this phone"	Not sent to other systems; only its SHA-256 is stored (as `customer_hash`)
customer ID	`auth.ts caller()`: `customer_hash = sha256("user:" + userId)` when signed in, else `sha256(deviceKey)`; also `user_id` uuid when signed in	64-char hex string (+ uuid)	application-generated (server)	per account (or per device before sign-in)	Local to One Local tables. A signed-in customer can read both their account-hash orders and their device-hash (pre-sign-in) orders (`matches`)
order ID (One Local)	`store.ts buildCheckout`: `crypto.randomUUID()` (server). SQL function path: `gen_random_uuid()`	uuid	application (checkout path) / DB (SQL function path)	global	`delivery_jobs.ol_order_id`, `ol_payment_events.order_id`, Razorpay `notes.checkout_id` (checkout id, not order id)
checkout ID	`store.ts buildCheckout`: `crypto.randomUUID()`	uuid	application	global	Razorpay `receipt` and `notes.checkout_id`; app uses it for `/checkouts/:id`

ID	Where generated	Type	DB- or app-generated	Uniqueness scope	Reused across systems?
order number (customer-visible, POS)	`ol_create_restaurant_order`: `next_org_seq(org,'order')`, `display_no = 'OL-' || lpad(n,5,'0')` e.g. `OL-00012`	int8 + text	DB (POS counter)	per organization	POS `orders.order_no / display_no`
order number (dispatch)	`dispatch.ts createJobForOrder`: `'OL-' + first 6 chars of ol_orders.id, uppercased` e.g. `OL-3F9A1B`. Stored on `delivery_jobs.order_no` only	text	application	effectively unique by chance (24 bits)	Shown to riders as `order_ref`. **Different format from the POS** `display_no` **that carries the same** `OL-` **prefix**
order number (customer UI)	`payments.ts checkoutOptions`: description `Order #<first 8 chars of checkout id, uppercased>`	text	application	–	Razorpay Checkout page only
payment ID	Razorpay `pay_…` → `ol_checkouts.razorpay_payment_id` (or `ol_orders` pre-0026); Razorpay order `order_…` → `razorpay_order_id`; Razorpay event id → `ol_payment_events.event_id`; refund `rfnd_…` → `ol_refunds.reference`	text	Razorpay	per Razorpay account	POS `payments.id` is a separate uuid (only for cash on delivered restaurant orders) with `reference = 'onelocal:<ol_orders.id>'`
rider ID	`riders.id`	uuid	DB default	global	`delivery_jobs.rider_id`, `delivery_assignments.rider_id`, `push_tokens.rider_id`; also `riders.user_id` → `auth.users`
KOT ID	`kot_tickets.id` uuid; `kot_no int8` from `next_org_seq(org,'kot')`	uuid + int8	DB	`kot_no` unique per organization	POS only
delivery ID	`delivery_jobs.id` uuid; `delivery_jobs.idempotency_key = 'onelocal:<ol_order_id>:delivery'`	uuid	DB	global; one per `ol_order_id` (UNIQUE)	Contract name in docs: "job" / "delivery job"
delivery code (OTP)	`dispatch.ts deliveryOtp`: 4 digits derived `HMAC-SHA256(secret, "delivery-otp:<ol_order_id>")`; only `otp_hash = sha256(code + ":" + ol_order_id)` stored	string	application	per order	**No code anywhere in this repo displays it to the customer or verifies it** (`deliveryOtp` is only called inside `createJob`; INFERRED that customer display and verification are future work)
request ID	dispatch service callers send `x-request-id`; server also builds ids like `order-flow:<order>`, `accept:<offer>`, `partner-accept:<pos order>`	text	caller/application	per call	Stored in `delivery_events.meta.request_id`
idempotency key (customer)	`frontend/app/(tabs)/cart.tsx`: `randomId(16)` per checkout attempt, header `Idempotency-Key` (regex `^[A-Za-z0-9_-]{8,128}$`)	string	client	per (customer_hash, key)	`ol_checkouts.idempotency_key` (or `ol_orders.idempotency_key` with suffix `:2` … before 0026)
POS idempotency ref	`orders.external_ref = 'onelocal:<ol_orders.id>'`	text	DB function	–	Links POS order back to One Local
push token	Expo `ExponentPushToken[...]`	text	Expo	unique per device	`ol_partner_devices.expo_push_token`, `push_tokens.expo_push_token`
Privacy reference	`privacy.ts newReference`: `<2-letter kind>-<yymmdd>-<6 chars>`	text	application	UNIQUE (draft table)	Shown to the customer
Product ID	demo `p1`; real `<store id>_<menu item uuid>[_<variant uuid>]`	text	seed / derived	per store	Used inside `ol_orders.items[].id` and `ol_order_items.product_id`

03 — ORDER / TRANSACTION LIFECYCLE (One Local / OL-ui)

Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**; **DRAFT** = migration not applied to production per `docs/JOURNAL.md`.

0. The most important finding, up front

This repo contains **two disconnected order creation paths**:

Path	Where	Creates	Reaches the restaurant / POS?
P1. Customer checkout (the real customer flow)	`frontend/server/store.ts createCheckout` ← `POST /api/checkouts` and `POST /api/payments/order`	`ol_checkouts` (if 0026), `ol_orders`, `ol_order_items` — **no** `organization_id`, **no** `pos_order_id`, **no** POS `orders` row	**No.** The Partner app lists orders `WHERE ol_orders.organization_id = <staff org>` (`restaurant.ts listOrders`), so a customer-checkout order never appears. Confirmed by the journal (2026-09-27 and 2026-09-28: "Orders for Master-approved restaurants are not yet sent on to the POS or the VINII Partner app", "Checkout still does not create the kitchen's side of an order")
P2. Restaurant order SQL function	`ol_create_restaurant_order` (0023), called **only** by `restaurant.ts createTestOrder` ← `POST /api/partner/test-order`	POS `orders`, `order_items`, `order_events`, and `ol_orders` (with `organization_id`, `pos_order_id`, `restaurant_deadline`)	Yes — this is the only path the Partner app and `ol_partner_act` state machine can act on

Consequences (all INFERRED from the code paths above): a customer's food order is saved and (for COD, or once paid) handed to the rider-dispatch pool, but no restaurant is ever notified; and the restaurant-side state machine (accept → ready → out → delivered) cannot be exercised by a real customer order. Status for non-POS-linked orders never advances past `placed` in this repo (nothing writes it), while the customer's tracking screen shows a client-side timed animation, not `ol_orders.status` (`frontend/app/order/[id].tsx`, `frontend/src/native/ios-extras.ts stageAt`).

Production status of tables/migrations used below is in `01_DATABASE.md` §0.

---

1. How an order starts

Question	Answer (with source)
Who/what creates it	The customer app's cart screen `frontend/app/(tabs)/cart.tsx handleCheckout`: `POST /api/checkouts` (cash) or `POST /api/payments/order` (online). Both call `createCheckout` in `frontend/server/store.ts`
Cart creation	Client-only. `frontend/src/cart.tsx` keeps **two carts** in AsyncStorage key `onelatur.cart.v2` — `food` (one restaurant only) and `shopping` (many stores). No server-side cart exists. The server sees the cart only as `items:[{id,qty}]` on `POST /api/cart/quote` and on checkout
Order ID generation	`crypto.randomUUID()` in `buildCheckout` — application-generated, one uuid per store order and one for the checkout
Customer identification	Headers `x-customer-key` (device) and optional `Authorization: Bearer <Supabase access token>`; `auth.ts caller()` → `customer_hash` + `user_id`. The **server accepts a device-key-only checkout** (`requireCustomer` needs only `who.owner`); the **app** forces sign-in first when Supabase is configured (`cart.tsx` pushes `/sign-in?next=checkout`)
Organization identification	**Not set** by checkout (P1). Set only by `ol_create_restaurant_order` (P2) from `p.organization_id`
Location / store identification	`ol_orders.store_id` = store of each bill group (needs 0026); `ol_orders.city_id` = `resolveCity(request)` (`?city=` or `x-city`, default `latur`; only `latur` is active in `ACTIVE_CITIES`)
Order splitting	One order **per store** in the cart (`pricing.ts quote` → `groups`). Food cart = exactly one restaurant (409 otherwise); shopping cart = any number of stores. Food and shopping can never be in the same checkout (409)
Pricing source	Server recomputes everything from the catalogue (`pricing.ts quote`); the app's `total` is only a ±₹0.50 sanity check (409 "Prices have changed")

2. Pricing, discounts, taxes (`frontend/server/pricing.ts`)

Rule	Value / behaviour
Item price	Catalogue `price` (`ol_products.price`, or `menu_item_variants.price` / `menu_items.price` for marketplace listings); MRP = price for marketplace listings
Delivery fee	Per store: ₹15 when that store's item total < ₹199, else ₹0 (`PRICING.deliveryFee`, `freeDeliveryFrom`). A duplicate helper `deliveryFee()` exists in `catalog.ts` (unused)
Handling fee, platform fee	Constants `0` ("undecided")
GST on restaurant food	5% added on top of food lines per store; split cgst/sgst; `gst_added.total` = food GST + GST on delivery (rate `0`, undecided) → stored as `ol_orders.gst`
GST on packaged goods	Included in MRP, backed out for display only (`gst_included`) using a sample per-category table (flagged "SAMPLE RATES — confirm with the CA"); **not stored** on the order
Coupon	`ol_coupons` (`kind flat|pct`, `value`, `min_order`, optional `store_id` from 0026). Applied to the checkout item total (or one store's part for a store coupon); discount spread across stores in paise, remainder on the biggest store. Checkout fails with 409 if coupon not ok
Store bill total	`max(item_total + delivery_fee + handling_fee + platform_fee + gst_added.total − discount, 0)`
Checkout total	Sum of store totals
Per-line discount	`ol_order_items.discount` exists but is never set (always 0)
Order-level cap/limit	`quote` ≤ 50 lines, qty 1–99; `createCheckout` rate limit 5 per 60 s per customer (`limit()` is per-process memory)

3. State transitions

Notation: EVENT / ACTOR / TABLE / FIELD / OLD → NEW / CODE LOCATION / API-RPC / SIDE EFFECTS. "Old value" `∅` = row does not exist yet.

3.A Customer checkout, cash on delivery (path P1)

A1. Cart priced — EVENT: quote requested. ACTOR: customer app. TABLE: none (no write). CODE: `frontend/server/pricing.ts quote`. API: `POST /api/cart/quote` (limit 120/min). SIDE EFFECTS: none; coupon check reads `getCatalog()`; `POST /api/coupons/apply` is a second, simpler coupon call (`applyCoupon(code, subtotal)`).

A2. Checkout created (COD) - EVENT: customer taps Pay on delivery. ACTOR: customer (device key or signed in). - API: `POST /api/checkouts` with `Idempotency-Key` header → `createCheckout(request, city, online=false)`; `POST /api/orders` is the legacy equivalent returning the first order. - TABLE `ol_checkouts` (only when 0026 applied, `checkoutsReady()`): row INSERT — `kind`, `item_total`, `discount`, `delivery_fee`, `total`, `coupon_code`, `payment_method='cod'`, `payment_status='not_charged'`, `idempotency_key`, `city_id`, `customer_hash`, `user_id`, `created_at`. OLD `∅` → NEW row. - TABLE `ol_orders`: one INSERT per store — `status = 'placed'`, `payment_method='cod'`, `payment_status='not_charged'`, `items` jsonb, `subtotal`, `discount`, `delivery_fee`, `gst`, `total`, `coupon_code`, `rider_name='Sample rider'`, `rider_phone=''`, `eta_minutes` (= parsed store delivery time or 10), `delivery_address` jsonb, `city_id`, `created_at`; with 0026: `checkout_id`, `store_id`; without 0026: `idempotency_key` (`key`, `key:2`, `key:3` …). OLD `∅` → NEW `placed`. - TABLE `ol_order_items`: INSERT one row per line (`order_id`, `product_id`, `name`, `weight`, `image`, `price`, `mrp`, `qty`). **Best effort** — failure is only `console.error`'d. INFERRED: the `product_id → ol_products(id)` FK rejects ids of marketplace products (they are not rows in `ol_products`), and since `ol_products` is empty after the clean-up, every line insert would fail in production; then `ol_order_items` stays empty. - Failure handling: if `ol_orders` insert fails, the `ol_checkouts` row is deleted; on unique-index conflict (code 23505) the first result is returned (idempotent). - Idempotency: unique `(customer_hash, idempotency_key)`; same key returns the earlier checkout. In-memory mode (no database) keeps replies in a Map for 24 h and rejects a changed fingerprint with 409. - SIDE EFFECTS: `dispatchOrders(order ids)` (see 3.D). No notification to customer, restaurant, or POS. No inventory change. No analytics write.

A3. Orders listed / opened — ACTOR: customer. `GET /api/orders` (`listOrders`: `customer_hash IN matches`, excludes `payment_status IN (pending,failed)`, 200 newest) and `GET /api/orders/:id`, `GET /api/checkouts/:id`. Read-only.

3.B Online payment (Razorpay) — requires 0026 (DRAFT)

B1. Payment started — ACTOR: customer. API: `POST /api/payments/order` → `payments.ts startPayment` → `createCheckout(..., online=true)`. TABLE `ol_checkouts` INSERT with `payment_method` = chosen (`upi|card|netbanking|wallet|paylater`) and `payment_status = 'pending'`; `ol_orders` INSERT with the same fields, `status='placed'`. Then `POST https://api.razorpay.com/v1/orders {amount: paise(total), currency:'INR', receipt: checkout id, notes:{checkout_id}}` (Basic auth with `RAZORPAY_KEY_ID:RAZORPAY_KEY_SECRET`); `ol_checkouts.razorpay_order_id` set (`updatePayment` allowed from `pending|failed`). Without Razorpay keys `features().online_payments` is false → 503 and the cart stays COD-only. Orders with `payment_status IN (pending,failed)` are hidden from the customer's list.

B2. Payment confirmed by the app — API: `POST /api/payments/verify {checkout_id, razorpay_order_id, razorpay_payment_id, razorpay_signature}`. Checks `HMAC-SHA256(key_secret, order_id|payment_id)`, then (best effort) `GET /v1/payments/<id>` to compare amount/order. TABLE `ol_checkouts`: `payment_status pending|failed|not_charged → paid`; `razorpay_payment_id`, `paid_at`, `payment_error=null`, `payme`

nt_method (the method Razorpay reports: upi, card, netbanking, wallet, paylater, emi, cardless_emi). Trigger ol_checkouts_payment_to_orders copies method/status/paid_at/error to each ol_orders row. SIDE EFFECTS: dispatchOrders(order_ids). Amount mismatch → payment_error='Paid amount did not match the order', 409.

B3. Payment confirmed/failed by Razorpay webhook (source of truth) — API: POST /api/payments/webhook; signature x-razorpay-signature = HMAC-SHA256(RAZORPAY_WEBHOOK_SECRET, raw body). Events handled: payment.captured, order.paid → same as B2 (markPaid); payment.failed → ol_checkouts.payment_status pending → failed, payment_error = Razorpay description; refund.processed → see 3.E. Dedupe by x-razorpay-event-id in ol_payment_events (insert after handling). Rate limit 600/min.

B4. Switch a failed/abandoned online payment to cash — API: POST /api/payments/cod {checkout_id}. ol_checkouts: payment_method → cod, payment_status pending|failed → not_charged. SIDE EFFECTS: dispatchOrders.

B5. Second payment on an already paid checkout — not auto-refunded; console.error asks the operator to refund from the Razorpay dashboard.

3.C Restaurant order, POS-linked (path P2) — ol_create_restaurant_order + ol_partner_act

Callable in this repo only from POST /api/partner/test-order (test order: 1–2 random available menu items, customer_hash='partner-test:<staff user id>', delivery_fee 0, payment fixed 'cod').

C1. Placed — ACTOR: server (service role). RPC ol_create_restaurant_order. Validations: OL_NOT_ACCEPTING unless ol_merchant_channels.accepting; items must exist for the org, menu_items.is_available, and not switched off in item_channel_status for channel='one_local' (off_until null or future); qty 1–99; ≤50 lines. TABLE orders INSERT: status 'new', channel 'online', placed_by 'integration', order_no/display_no (OL-00012 via next_org_seq(org,'order')), subtotal, gst_pct 5, gst_amount, total = subtotal + gst + delivery_fee, external_ref 'onelocal:<ol id>'. order_items INSERT (status pending). order_events INSERT kind 'placed', actor 'one_local'. ol_orders INSERT: status 'placed', payment_method 'cod', payment_status 'not_charged' (default), restaurant_deadline = now() + accept_window_seconds (default 60 s), pos_order_id, organization_id, items jsonb, gst. SIDE EFFECT (test-order route only): Expo push "New order OL-xxxxx … accept within 60 seconds" to ol_partner_devices of the org (push.ts pushToRestaurant). **Does not call dispatch, does not insert** ol_order_items. The function hard-codes payment_method 'cod'.

C2. Accept — ACTOR: restaurant staff (roles org_owner, org_admin, manager, biller, location_manager, operations_manager). API POST /api/partner/orders/:posOrderId/accept {version, prep_minutes} → RPC ol_partner_act(... 'accept'). Preconditions: orders.status='new'; not past restaurant_deadline + 5 s (OL_EXPIRED); prep 5–120; p_expected = orders.updated_at (OL_STALE otherwise). Writes: kot_tickets INSERT (kot_no via next_org_seq(org,'kot'), station 'main', status 'new'); order_items.kot_ticket_id set (items with status pending); orders.status new → sent_to_kitchen; ol_orders.status placed → accepted, prep_minutes, eta_minutes = prep + 15; ol_merchant_channels.missed_in_a_row → 0; order_events accepted + kot_fired. SIDE EFFECT: dispatch.ts onRestaurantAccepted → releases a held delivery job (only if one exists and dispatch_trigger='accepted').

C3. Reject — same roles. POST …/reject {version, reason}. Precondition orders.status='new', reason required. orders.status new → cancelled; all order_items.status → cancelled; ol_orders.status placed → cancelled, cancel_reason = 'Restaurant: <reason>'; missed_in_a_row → 0; order_events 'rejected'. SIDE EFFECT: onRestaurantRejected → dispatch_cancel the job.

C4. Timeout (restaurant did not answer) — ACTOR: system. Triggered by RPC ol_expire_unanswered, which runs at the start of every Partner GET /api/partner/orders (and nowhere else — no scheduler in this repo). In ol_partner_act('timeout') when orders.status='new' and now() > restaurant_deadline: orders.status → cancelled; items cancelled; ol_orders.status placed → cancelled, cancel_reason 'The restaurant did not respond in time'; ol_merchant_channels.missed_in_a_row +1; when ≥ auto_pause_after (default 3) → accepting=false (auto-pause); order_events 'timed_out'. Dispatch job cancellation follows in the next dispatch sweep (order_cancelled).

C5. Ready — POST …/ready. Precondition orders.status='sent_to_kitchen'. kot_tickets.status (new|preparing) → ready; order_items.status (pending|preparing) → ready; orders.status → ready; ol_orders.status accepted → packed; order_events 'kot_ready'. Roles also include kitchen.

C6. Out for delivery — POST …/out. Precondition orders.status='ready' AND ol_orders.status='packed'. kot_tickets.status → delivered; order_items.status → delivered (non-cancelled); orders.updated_at only (**POS** orders.status **stays** ready); ol_orders.status packed → out; order_events 'out_for_delivery'.

C7. Delivered (cash collected) — POST …/delivered. Precondition ol_orders.status='out'. orders.status → delivered, payment_method 'cash', paid_at = now(); payments INSERT (method 'cash', amount = orders.total, reference 'onelocal:<ol id>', created_by = staff user id); ol_orders.status → delivered, payment_status not_charged → paid; order_events delivered and paid. No dispatch hook is called here. The staff-marked "delivered" is independent of delivery_jobs.status (INFERRED: two parallel delivery truths).

3.D Rider dispatch (0029, DRAFT; production behaviour = no-op until applied and DISPATCH_SIGNING_SECRET set)

Hook: dispatchOrders(orderIds) runs after a COD checkout (3.A2), after markPaid (3.B2/B3) and after switch-to-cash (3.B4). If DISPATCH_SIGNING_SECRET/DISPATCH_OTP_SECRET is unset it only warns once and returns; if it throws it is logged and the sweep retries within 24 h (delivery_orders_needing_rider). "Successful order" = payment_method='cod' OR payment_status='paid' (createJobForOrder throws 409 order_not_deliverable otherwise).

EVENT	ACTOR	TABLE.FIELD	OLD → NEW	CODE / RPC	SIDE EFFECTS
Job created	server (order flow) or dispatch service over HTTP	`delivery_jobs` INSERT; `status`	∅ → `created`; `version 1`; `dispatch_trigger` (org setting else `'placed'`); `otp_hash`; pickup point from `ol_stores.lat/lng` else `ol_merchant_channels.lat/lng` (422 `no_pickup_location` otherwise); drop point from `ol_orders.delivery_address.lat/lng` (400 if absent; 422 `outside_delivery_area` if outside `SERVICE_AREAS`)	`dispatch.ts createJob` → RPC `dispatch_create_job`; HTTP `POST /api/dispatch/jobs` (service-signed)	`delivery_events` row `delivery.created`; customer name/phone read from `auth.admin.getUserById`; cash to collect = `ol_orders.total` for COD else 0
Job released	server	`delivery_jobs.released_at`, `rounds_base`, `version +1`; `delivery_events`	`released_at NULL → now()`	RPC `dispatch_release` (immediately when trigger `'placed'`; after restaurant accept when `'accepted'`)	event `delivery.released`
Offered to rider	system	`delivery_assignments` INSERT (`outcome NULL`, `expires_at = now + offer_seconds`, default 30 s); `delivery_jobs.status`	`created → offered`	RPC `dispatch_offer_next`; candidate: `riders.status='approved'`, online, `last_seen_at` < 60 s, vehicle match, no live offer/active job, never offered this job, within radius (default 8 km); order: linked store/org first, nearest, fewest jobs today	event `delivery.offered`; Expo push "New delivery job" (`pushToRider`, ttl 30 s); `acceptance='auto'` riders are auto-accepted at once
Rider accepts	rider	`delivery_assignments.outcome`; `delivery_jobs.status`, `rider_id`	`NULL → accepted`; `offered → assigned`	`POST /api/rider/offers/:id/accept` → RPC `dispatch_answer`	event `delivery.assigned`
Rider declines	rider	`delivery_assignments.outcome`	`NULL → rejected`	`POST /api/rider/offers/:id/decline` → `dispatch_answer` then `advanceJob`	event `delivery.rejected`
Offer lapses	system	`delivery_assignments.outcome`	`NULL → timed_out`	`dispatch_offer_next` (called by sweep, heartbeat `advanceDue`, rider reading offers, rider going online)	event `delivery.timed_out`
No rider	system	`delivery_jobs.escalated_at`, `status`	→ `created` with `escalated_at` set after `max_rounds` offers (or `max_rounds × offer_seconds` with nobody online)	`dispatch_offer_next`	event `delivery.no_rider`; push "No rider found yet" to the restaurant devices
Cancelled	restaurant reject / sweep sees `ol_orders.status='cancelled'` / service call	`delivery_jobs.status`, `cancel_reason`; live `delivery_assignments.outcome → withdrawn`	→ `cancelled` (refused when `picked_up|at_drop|delivered`)	RPC `dispatch_cancel`; HTTP `POST /api/dispatch/jobs/:id/cancel` (reasons `restaurant_rejected, restaurant_timed_out, customer_cancelled, other`)	event `delivery.cancelled`; push "Order cancelled" to the rider
At pickup / picked up / at drop / delivered / exception	rider app (**not in this repo**)	`delivery_jobs.status` allows these values	–	**No function or route in this repo sets them** (journal: "the job stops at assigned")	–

Customer-facing: `GET /api/dispatch/jobs?ol_order_id=` returns `jobView` for services (no rider phone); **no customer route exposes it**; the customer app shows `rider_name 'Sample rider'`.

3.E Refunds

EVENT	ACTOR	TABLE.FIELD	OLD → NEW	CODE / API	SIDE EFFECTS
Gateway refund recorded	Razorpay (webhook `refund.processed`)	`ol_refunds` INSERT (`status 'processed'`, `method 'original'`, `reason 'Refunded through Razorpay'`, `reference = rfnd id`, booked on the first store order it fits); `ol_orders.payment_status` → `refunded` when that order's processed refunds ≥ its total; `ol_checkouts.payment_status` → `refunded` when all ≥ checkout total	`paid → refunded`	`store.ts recordGatewayRefund` (found by `razorpay_payment_id`)	Does **not** change `ol_orders.status`, does not touch POS, does not release inventory
Internal refund	support tooling (**no route or caller exists**)	`ol_refunds` INSERT (`status 'processed'`), `ol_order_items.refunded_qty += ceil(amount/price)`, `ol_orders.payment_status → refunded` when total covered	allowed only when `ol_orders.status IN (delivered,out,packed,accepted)`	RPC `ol_process_refund` via `store.ts refundOrder`	Records only; no money movement call. Statuses `requested/approved/rejected` exist but nothing creates them

EVENT	ACTOR	TABLE.FIELD	OLD → NEW	CODE / API	SIDE EFFECTS
Customer-requested refund	–	–	–	**none** (no UI or route)	–

3.F Cancellation and failure matrix

Case	Supported?
Customer cancels an order	**No route, no UI.** `ol_orders.cancelled_by` is never written
Restaurant rejects / times out	Yes, only for POS-linked orders (3.C3/C4)
Online payment fails	`ol_checkouts.payment_status → failed`; order stays `placed` but hidden from the list; customer can switch to cash
Admin cancels an order	**No** admin order routes exist (`/api/admin/*` covers listings, shoppers, media, demand, organizations)
Delivery job cancelled	Yes (3.D)

3.G Everything else asked for

Topic	Current state
Order acceptance	Only POS-linked orders (3.C2). Customer-checkout orders: **UNKNOWN who accepts** — nothing does in this repo
Preparation / ready	POS-linked only (3.C5). The SQL function copies `station` and `course` from `menu_items` to `order_items`; `menu_items.prep_minutes` is not used. ETA = the accepted prep time + 15 (`ol_orders.eta_minutes`)
Fulfilment / delivery / completion	Staff-marked (3.C6–C7) for POS-linked; rider app for job states (not in repo). For customer-checkout orders `ol_orders.status` is never advanced → **stays** `placed` (INFERRED)
Inventory impact	**None.** Availability only: catalogue excludes `ol_products.is_available=false` (demo) and POS items with `menu_items.is_available=false` or switched off for `one_local`. `quote` re-checks only that the product id exists in the cached catalogue (30 s TTL)
Notifications	Expo push to restaurant devices (`new_order` — **only from the test-order route**; `no_rider`), to riders (`job_offer`, `job_assigned`, `job_cancelled`). No customer push, SMS, WhatsApp or email in this repo. iOS Live Activity/widget are driven locally by the client timer (`frontend/src/native/ios-extras.ts`)
Analytics	None stored. Console log lines: `[admin] {json}` audit line for admin writes, `[privacy] {json}`, `[dispatch]`, `[payments]`, `[api]` errors. The admin "demand" report counts `launch:` rows in `ol_support_requests`
Customer delivery code	Derived (`dispatch.ts deliveryOtp`), only its hash is stored; **no endpoint returns it to the customer and nothing verifies it** (INFERRED unfinished)
Loyalty / coins / wallet	Not implemented (feature flags false; wallet screen demo in AsyncStorage)
Invoices	None generated
Tables / dine-in	Not used (`orders.table_id` always NULL, channel `'online'`)

04 — API MAP and DATABASE OPERATIONS (One Local / OL-ui)

Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**; **DRAFT** = table/function from a migration not applied to production per `docs/JOURNAL.md`.

0. How the API is built

- Expo Router API routes in `frontend/app/api/**/+api.ts` (one file per route/method); each is a thin wrapper around a function in `frontend/server/*.ts`. Customer and Partner apps (`frontend/`, `partner/`) call the same server; the Partner app uses the `/api/partner/*` prefix.
- There is **no Edge Function, no Supabase Database Webhook, no direct SQL from app code, and the apps never call** `supabase.from(...)` — `grep` finds `supabase.auth.*` only in the client (sign-in). All reads/writes go through `db()` in `frontend/server/db.ts` = `@supabase/supabase-js` with `SUPABASE_URL` + `SUPABASE_SERVICE_ROLE_KEY` (service role; bypasses RLS).
- Every state-changing multi-table operation is an RPC (Postgres function). Errors are `OL_*` exceptions mapped to HTTP status by `db.ts fromDbError` (`OL_BAD_REQUEST/BAD_QTY/EMPTY 400`, `NOT_FOUND/NOT_ONE_LOCAL 404`, `NOT_ACCEPTING/ITEM_UNAVAILABLE/STALE/WRONG_STATE/EXPIRED/NOT_DELIVERABLE/PICKED_UP/TAKEN 409`, `OL_GONE 410`) plus a machine `code` for the dispatch/rider APIs.
- Without `SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY` the server runs in an **in-memory fallback** (`store.ts`, `privacy.ts`, `marketplace.ts` memory, `catalog.json`), used by `expo start` for UI work.
- Auth schemes: **(C)** customer: `x-customer-key` header (device) and optional Supabase JWT (`auth.ts caller()`); **(P)** partner staff: Supabase JWT + `org_users` row (`partner.ts requirePartnerAuth`); **(R)** rider: JWT + approved `riders` row (`rider.ts requireRider`); **(A)** platform admin: JWT + `platform_admins` row (`admin.ts requireAdmin`); **(S)** service: bearer `DISPATCH_SERVICE_TOKEN` + `x-timestamp` + `x-signature` HMAC + `x-request-id` (`dispatch.ts serviceRequest`); **(W)** Razorpay webhook signature; **(–)** none.
- City scoping: `resolveCity` reads `?city=` or `x-city` (default `latur`); `404` if not in `ACTIVE_CITIES` (hardcoded `{latur}`).
- Rate limiting: in-process memory (`http.ts limit`), resets on restart, not shared across servers.

1. Route table

Method + path	Auth	Handler	Tables / RPCs / external	Notes
Catalogue (reads)				
GET `/api/catalog`	–	`catalog.ts getCatalog`	`ol_categories, ol_brands, ol_stores, ol_products(is_available), ol_banners, ol_discover_chips, ol_coupons(active), ol_events, ol_themed_stores, ol_reels`; plus `organizations, menu_categories, menu_items, menu_item_variants, item_channel_status(channel='one_local')` via `marketplace.ts readItemMaster` for live listings	30 s in-memory cache; invalidated by admin writes
GET `/api/categories`, `/api/banners`, `/api/discover`, `/api/events`, `/api/reels`, `/api/coupons`	–	same cache	as above	
GET `/api/products`, `/api/products/:id`, `/api/products/:id/story`, `/api/products/buy-again`	–	`catalog.ts filterProducts / findProduct`	cache	`buy-again` is a hard-coded sample id list `p4,p6,p8,p11,p16`
GET `/api/stores`, `/api/stores/:id`, `/api/stores/:id/products`	–	cache	cache	
GET `/api/media/:id`	–	`media.ts serveMedia`	`data/media.json` (key → Storage URL, 302); dev: local `backend/.media_build`	
GET `/api/health`, `/api`	–	static	–	
GET `/api/features`	–	`features.ts`	env only	flags `ai_search, assistant, maps, online_payments, wallet, coins, google_sign_in`
Location				
GET `/api/serviceability?lat&lng`	–	`catalog.ts serviceability`	constant `SERVICE_AREAS`	

Method + path	Auth	Handler	Tables / RPCs / external	Notes
GET `/api/coverage?lat&lng`	–	`coverage.ts`	`ol_service_areas` (60 s cache) + `geo.ts` India places file	levels `live|discover|soon`
GET `/api/nearby?lat&lng`	–	`nearby.ts`	Google Places (New) or Overpass (OSM)	limit 20/min
GET `/api/places/autocomplete`, `/api/places/:id`, `/api/geocode/reverse`	–	`maps.ts`	Google Places / Geocoding	needs `GOOGLE_MAPS_API_KEY`
GET `/api/india/popular`, `/api/india/search`, `/api/india/states[/:state[/:district]]`	–	`geo.ts`	bundled `data/india-places.tsv.gz`	
Cart and checkout				
POST `/api/cart/quote`	(C) optional	`pricing.ts quote`	cache reads only	limit 120/min
POST `/api/coupons/apply`	–	`catalog.ts applyCoupon`	cache	limit 10/min
POST `/api/checkouts`	(C)	`store.ts createCheckout`	`ol_checkouts` INSERT (0026), `ol_orders` INSERT, `ol_order_items` INSERT; reads `ol_checkouts` / `ol_orders` for idempotency; then `dispatch.ts dispatchOrders`	header `Idempotency-Key`; limit 5/60 s per customer
GET `/api/checkouts/:id`	(C)	`store.ts getCheckout`	`ol_checkouts`, `ol_orders`	owner only (`customer_hash IN matches`)
POST `/api/orders`	(C)	`store.ts createOrder`	as `/checkouts`, returns first order	legacy for older app versions
GET `/api/orders`	(C)	`store.ts listOrders`	`ol_orders` (`customer_hash IN matches`, `payment_status NOT IN (pending,failed)`, 200 newest)	
GET `/api/orders/:id`	(C)	`store.ts getOrder`	`ol_orders`	
POST `/api/booking-enquiries`	(C) optional	`store.ts createEnquiry`	`ol_booking_enquiries` INSERT	validates event/slot/date (≤30 days)/phone regex; limit 5/h
POST `/api/support-requests`	(C) optional	`store.ts createSupportRequest`	`ol_support_requests` INSERT	limit 5/h
POST `/api/launch-requests`	(C) optional	`store.ts createLaunchRequest`	`ol_support_requests` INSERT (`topic 'other'`, `reference 'launch:<place id>'`)	limit 3/day
Payments (Razorpay)				
POST `/api/payments/order`	(C)	`payments.ts startPayment`	`createCheckout(online)` writes (above); `ol_checkouts` UPDATE `razorpay_order_id`; **Razorpay** `POST /v1/orders`	503 if no keys / 0026 not applied
POST `/api/payments/verify`	(C)	`payments.ts verifyPayment`	`ol_checkouts` UPDATE (→ `paid`), trigger → `ol_orders`; Razorpay `GET /v1/payments/:id`; then `dispatchOrders`	HMAC verification
POST `/api/payments/cod`	(C)	`payments.ts switchToCash`	`ol_checkouts` UPDATE (→ `cod` / `not_charged`); `dispatchOrders`	
POST `/api/payments/webhook`	(W)	`payments.ts handleWebhook`	`ol_payment_events` SELECT/INSERT; `ol_checkouts` UPDATE; `ol_refunds` INSERT/UPDATE; `ol_orders` UPDATE (`payment_status`); `dispatchOrders`	events `payment.captured, order.paid, payment.failed, refund.processed`
GET `/api/payments/checkout-page`	(C)	`payments.ts checkoutPage`	`ol_checkouts` (read)	Returns an HTML page that loads `https://checkout.razorpay.com/v1/checkout.js` inside the app's WebView
Privacy (DPDP)				

Method + path	Auth	Handler	Tables / RPCs / external	Notes
GET `/api/privacy/export`	(C) signed-in	`privacy.ts exportMyData`	`ol_orders, ol_support_requests, ol_booking_enquiries, ol_refunds, ol_checkouts?, ol_consents?, ol_privacy_requests?` ; Supabase `auth/v1/user` ; writes `ol_privacy_requests` INSERT (kind export)	limit 5/h
GET/POST `/api/privacy/consents`	(C) signed-in	`recordConsents`, `currentConsents`	`ol_consents` INSERT/SELECT (0027 DRAFT; else "phone only")	
POST `/api/privacy/erasure`	(C) signed-in	`requestErasure`	UPDATE `ol_orders` (address NULL, user_id NULL, `customer_hash = erased:<uuid>` , key NULL), UPDATE `ol_checkouts` , UPDATE `ol_refunds.requested_by` , DELETE `ol_booking_enquiries` , UPDATE `ol_support_requests` (message erased), INSERT `ol_consents` (optional ones withdrawn), `ol_privacy_requests` INSERT/UPDATE; Supabase `auth/v1/logout?scope=global` or `auth.admin.deleteUser` (only if `PRIVACY_HARD_DELETE=on` and no other membership in `org_users, riders, platform_admins`)	refused (409) while an order placed in the last 24 h is still open
POST `/api/privacy/grievances` , `/api/privacy/nominee`	(C) signed-in	`raiseGrievance`, `nominate`	`ol_privacy_requests` INSERT (0027) or `ol_support_requests` (`reference 'privacy:<ref>'`)	
VINII Partner (restaurant staff)				
GET `/api/partner/me`	(P)	`partner.ts` + `restaurant.ts getOutlet`	`org_users` (+ `roles` , `organizations` joins), `ol_merchant_channels`	
GET `/api/partner/orders?tab=new|preparing|ready|past`	(P)	`restaurant.ts listOrders`	RPC `ol_expire_unanswered` ; SELECT `ol_orders` ⋈ `orders` ⋈ `order_items` filtered by `organization_id`	tab→status: new= `placed` , preparing= `accepted` , ready= `packed,out` , past= `delivered,cancelled` (3 days, 50)
GET `/api/partner/orders/:id`	(P)	`restaurant.ts getOrder`	`ol_orders` ⋈ POS, `order_events`	`:id` is the **POS order id**
POST `/api/partner/orders/:id/accept | reject | ready | out | delivered`	(P) with role action check	`restaurant.ts act`	SELECT `orders.organization_id` (ownership); RPC `ol_partner_act` ; then `dispatch.ts onRestaurantAccepted/Rejected`	body `{version = orders.updated_at, prep_minutes | reason}`
PATCH `/api/partner/outlet`	(P)	`restaurant.ts setOutlet`	UPDATE `ol_merchant_channels` (`accepting` , `back_at` , `missed_in_a_row=0` , `updated_by`)	
GET `/api/partner/items` ; PATCH `/api/partner/items/:id`	(P)	`listItems` , `setItem`	SELECT `menu_categories, menu_items, item_channel_status` ; UPSERT `item_channel_status` (`channel 'one_local'` ; `onConflict organization_id,menu_item_id,channel` , fallback update-then-insert)	
PUT `/api/partner/devices`	(P)	`registerDevice`	UPSERT `ol_partner_devices` on `expo_push_token`	
POST `/api/partner/test-order`	(P)	`createTestOrder`	RPC `ol_create_restaurant_order` ; Expo push	The only caller of that RPC
Rider (One Rider app)				
POST `/api/rider/availability`	(R)	`rider.ts setAvailability`	UPSERT `rider_availability` ; may call `advanceDue`	must be inside `SERVICE_AREAS` to go online
POST `/api/rider/heartbeat`	(R)	`heartbeat`	UPDATE `rider_availability` ; `advanceDue`	12/min
PUT `/api/rider/push-token`	(R)	`registerPushToken`	UPSERT `push_tokens`	
GET `/api/rider/offers/current`	(R)	`currentOffer`	`delivery_assignments` , `delivery_jobs` , `rider_availability` ; may `advanceJob`	204 when none
POST `/api/rider/offers/:id/accept | decline`	(R)	`acceptOffer` , `declineOffer`	RPC `dispatch_answer`	
GET `/api/rider/jobs/current` , `/api/rider/job`	(R)	`currentJob` , `jobForRider`	`delivery_jobs` (filtered `rider_id = me`)	privacy-shaped view `riderJob`

Method + path	Auth	Handler	Tables / RPCs / external	Notes
`s/:id`				
Dispatch service API				
POST `/api/dispatch/jobs` ; GET `/api/dispatch/jobs?ol_order_id=`	(S)	`dispatch.ts createJob / jobViewByOrder`	RPC `dispatch_create_job` etc.	
GET `/api/dispatch/jobs/:id`	(S)	`jobView`	`delivery_jobs, delivery_assignments, delivery_events, riders, rider_availability`	
POST `/api/dispatch/jobs/:id/release` , `/cancel`	(S)	`releaseJob` , `cancelJob`	RPC `dispatch_release` , `dispatch_cancel`	
POST `/api/dispatch/sweep`	(S)	`dispatch.ts sweep`	RPC `dispatch_due_jobs` ; `delivery_jobs` , `ol_orders` , view `delivery_orders_needing_rider`	**UNKNOWN who calls it** — journal says "a scheduler must call the sweep every 15 to 30 seconds"; `.emergent/cron/` holds only the generic Emergent cron watcher, no entry for it
Master admin API (Proviyaa OS console)				
GET `/api/admin/me` , `/organizations` , `/demand` , `/shoppers`	(A) read	`admin.ts`	`platform_admins` , `organizations` , `locations` , item master tables, `ol_stores` , `ol_merchant_channels` , `ol_support_requests` , `ol_orders` (counts), `auth.admin.listUsers`	page size 50, `X-Total-Count`
GET/POST `/api/admin/listings` ; GET/PATCH `/api/admin/listings/:id` ; GET `/…/preview` ; POST `/…/status`	(A) read / write (`master_admin, onboarding_admin, operations`)	`createListing, updateListing, setListingStatus, previewListing`	INSERT/UPDATE `ol_stores` (`status` , `approved_by` , `approved_at` , `created_by` , `updated_by` , …), INSERT/UPDATE `ol_merchant_channels` ; requires 0028 else 503 `migration_required`	going `live` needs ≥ 1 item available on One Local and no other live listing for that outlet
POST `/api/admin/shoppers/:id/block`	(A) write	`blockShopper`	`auth.admin.updateUserById(ban_duration '876000h'|'none')` ; refuses platform admins and merchant staff	
POST `/api/admin/media/upload-url` ; PUT/GET `/api/admin/media/dev/:key`	(A) write	`uploadUrl` , `devUpload/devFile`	Storage `createSignedUploadUrl` in bucket `onecity-catalog`	dev route works only without a database
Switched off				
`/api/assistant/*` , `/api/search/*`	–	static 503	–	"AI feature switched off while it moves to the new backend"

2. Database operations (important ones)

File : function	Table	Op	Fields written	Fields read	Condition	Trigger (user action)
`store.ts : createCheckout`	`ol_checkouts`	SELECT	–	`*`	`customer_hash` , `idempotency_key`	Pay tapped (retry safe)
`store.ts : createCheckout`	`ol_checkouts`	INSERT	id, customer_hash, user_id, kind, item_total, discount, delivery_fee, total, coupon_code, payment_method, payment_status, idempotency_key, city_id, created_at	–	0026 present	Pay tapped
`store.ts : createCheckout`	`ol_orders`	INSERT (batch)	id, customer_hash, user_id, items, subtotal, discount, delivery_fee, gst, total, coupon_code, payment_method, payment_status, status `'placed'` , rider_name, rider_phone, eta_minutes, delivery_address, city_id, created_at, checkout_id, store_id (or idempotency_key pre-0026)	–	–	Pay tapped
`store.ts : createCheckout`	`ol_checkouts`	DELETE	–	–	rollback if orders insert fails	–

File : function	Table	Op	Fields written	Fields read	Condition	Trigger (user action)
`store.ts : createCheckout`	`ol_order_items`	INSERT (batch)	order_id, product_id, name, weight, image, price, mrp, qty	–	best effort	Pay tapped
`store.ts : checkoutsReady`	`ol_checkouts`	SELECT id LIMIT 1	–	id	probe every 60 s until present	any checkout
`store.ts : listOrders / getOrder / ordersOf`	`ol_orders`	SELECT	–	`*`	`customer_hash IN matches` ; `payment_status NOT IN (pending,failed)`	My Orders / order screen
`store.ts : findPayment`	`ol_checkouts` , `ol_orders`	SELECT	–	id, customer_hash, total, payment_, *razorpay_*	by id / razorpay ids	payment calls
`store.ts : updatePayment`	`ol_checkouts`	UPDATE	any of payment_method, payment_status, razorpay_order_id, razorpay_payment_id, paid_at, payment_error	id	`payment_status IN (from)` — compare-and-set; trigger copies to `ol_orders`	verify / webhook / cod switch
`store.ts : recordGatewayRefund`	`ol_refunds` , `ol_orders` , `ol_checkouts`	SELECT/INSERT/UPDATE	refund row (amount, reason, method, status, reference, created_at, processed_at); `payment_status='refunded'`	order totals, refunds	`reference` unique (23505 ignored)	Razorpay `refund.processed`
`store.ts : refundOrder`	RPC `ol_process_refund`	RPC	see 01	–	–	**not called anywhere**
`store.ts : createEnquiry / createSupportRequest / createLaunchRequest`	`ol_booking_enquiries` , `ol_support_requests`	INSERT	see 01	–	–	Book It form / Help / "Launch One Local here"
`payments.ts : alreadyHandled / remember`	`ol_payment_events`	SELECT / INSERT	event_id, event, checkout_id, order_id, razorpay_payment_id	event_id	23505 ignored	webhook
`restaurant.ts : listOrders / counts / getOrder`	`ol_orders` (+ embedded `orders` , `order_items`), `order_events`	SELECT	–	status, totals, items, delivery_address, restaurant_deadline, POS display_no/status/updated_at/customer_name/phone/notes	`organization_id = staff org`	Partner screens (poll 5 s / 10 s)
`restaurant.ts : act`	`orders`	SELECT	–	organization_id	ownership check	accept/reject/ready/out/delivered
`restaurant.ts : act`	RPC `ol_partner_act`	RPC	across POS + OL tables (see 03 §3.C)	–	optimistic version	same
`restaurant.ts : setOutlet / getOutlet`	`ol_merchant_channels`	UPDATE / SELECT	accepting, back_at, missed_in_a_row, updated_by	all	org	Open/Closed switch
`restaurant.ts : listItems / setItem`	`menu_categories` , `menu_items` , `item_channel_status`	SELECT / UPSERT	item_channel_status (organization_id, menu_item_id, channel `one_local` , is_available, off_until, updated_at)	–	org ownership checked on `menu_items`	Partner "Items" screen
`restaurant.ts : registerDevice`	`ol_partner_devices`	UPSERT	organization_id, user_id, expo_push_token, platform, last_seen_at	–	–	Partner app start
`push.ts : pushToRestaurant / pushToRider`	`ol_partner_devices` , `push_tokens`	SELECT, DELETE (dead tokens `DeviceNotRegistered`)	–	expo_push_token	–	order/job events

File : function	Table	Op	Fields written	Fields read	Condition	Trigger (user action)
`dispatch.ts : createJobForOrder`	`ol_orders`, `ol_stores`, `ol_merchant_channels`, `auth.users` (admin API)	SELECT	–	order row, store row (name, lat, lng), channel (address, lat, lng, default_prep_minutes), user metadata	–	after a successful order
`dispatch.ts : createJob/advance/release/cancel/sweep`	RPCs + `delivery_jobs, delivery_assignments, delivery_events, riders, rider_availability, ol_orders, delivery_orders_needing_rider`	RPC / SELECT	via functions	–	–	see 03 §3.D
`rider.ts`	`riders, rider_availability, push_tokens, delivery_assignments, delivery_jobs`	SELECT / UPSERT / UPDATE	availability, push token	–	`rider.status='approved'`	Rider app
`catalog.ts : loadFromDb`	10 catalogue tables + item master	SELECT	–	all	cached 30 s	any browse
`marketplace.ts : readItemMaster / readListingRows / readChannels / readLocations`	`organizations, menu_categories, menu_items, menu_item_variants, item_channel_status, ol_stores, ol_merchant_channels, locations`	SELECT (paged 1000)	–	–	–	catalogue + Master
`admin.ts : createListing / updateListing / setListingStatus / saveChannel`	`ol_stores`, `ol_merchant_channels`	INSERT / UPDATE	see 04 §1	–	–	Master actions
`admin.ts : platformAdmin / isPlatformAdmin / isMerchantStaff`	`platform_admins`, `org_users`	SELECT	–	role	–	admin auth (30 s cache)
`admin.ts : orderCounts`	`ol_orders`	SELECT paged	–	id, user_id, checkout_id	`user_id IN (...)`	Master shoppers list
`admin.ts : demand`	`ol_support_requests`	SELECT	–	reference, message, created_at	`reference LIKE 'launch:%'`	Master demand screen
`privacy.ts`	see §1	SELECT/INSERT/UPDATE/DELETE	–	–	–	Privacy screens
`coverage.ts : liveAreas`	`ol_service_areas`	SELECT	–	city_id, name, lat, lng, radius_km	`active`	location check
`frontend/scripts/seed-catalog.mjs`	`ol_*` catalogue	UPSERT	demo rows (`is_demo=true`)	–	local DB only	developer command

3. Client-side (not DB) calls

Caller	Remote	Purpose
`frontend/src/auth.tsx`	Supabase Auth: `signUp`, `signInWithPassword`, `signInWithOtp` / `verifyOtp` (phone), `signInWithOAuth` (Google, PKCE), `getSession`, `onAuthStateChange`; `GET {SUPABASE_URL}/auth/v1/settings`	customer sign-in
`partner/src/auth.tsx`	`signInWithPassword`, session listener	staff sign-in
`frontend/src/api.ts`, `partner/src/api.ts`	`{BACKEND}/api/...` with bearer token + `x-customer-key`	all data
`frontend/src/components/razorpay-checkout.tsx`	WebView on `/api/payments/checkout-page`	payment UI
`frontend/src/native/ios-extras.ts`, `modules/onecity-live-activity`, `targets/widget`	local iOS APIs	Live Activity/widget (client timer, no server)

Legacy: `backend/*.py` (FastAPI media store + seed scripts) and `/api/assistant`, `/api/search` stubs are the **replaced** Python backend (`docs/BACKEND_REARCHITECTURE_PLAN.md`); `backend/` has no server entry file in the repo (INFERRED: remnants).

05 — EVENT MAP (One Local / OL-ui)

Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**; **DRAFT** = not applied to production per `docs/JOURNAL.md`.

0. Summary

- **There is no message queue, no Supabase Realtime subscription, no Database Webhook, no Edge Function and no pub/sub in this repo.** Phones **poll**.
- Two append-only event stores exist: POS `order_events` (written by the SQL functions of 0023) and `delivery_events` (the dispatch outbox, 0029 DRAFT). **Nothing in this repo reads either as a feed** for another system (`order_events` is read for the Partner order detail; `delivery_events` for the tracking `timeline`).
- `docs/PLATFORM_FOUNDATIONS.md` D20 / §6.5 describes a future outbox `ol_order_events` with canonical names (`checkout.created`, `order.created`, `order.accepted`, `order.cancelled`, `order.fulfilled`, `payment.*`…). **No migration creates it and no code writes it.**

1. Event table

EVENT | PRODUCER | CONSUMER | PAYLOAD | DATABASE EFFECT

1.1 Database triggers

EVENT	PRODUCER	CONSUMER	PAYLOAD	DB EFFECT
`BEFORE UPDATE` on `ol_orders`, `ol_merchant_channels`, `ol_checkouts`, `ol_stores`, `ol_privacy_requests`, `riders`, `org_dispatch_settings`, `delivery_jobs`	Postgres trigger	same row	–	`updated_at = now()` (`set_updated_at`)
`AFTER UPDATE OF payment_method, payment_status, paid_at, payment_error` on `ol_checkouts` (only when one changed)	trigger `ol_checkouts_payment_to_orders` (0026 DRAFT)	`ol_orders` of that checkout	new row values	UPDATE `ol_orders` set the same four fields
`BEFORE UPDATE OR DELETE` on `ol_consents`	trigger `ol_consents_append_only` (0027 DRAFT)	–	–	raises `OL_BAD_REQUEST` (DELETE allowed only with `set local ol.privacy_purge='on'`)

1.2 Outbox rows (written in the same transaction as the change)

`delivery_events` (DRAFT 0029) — `kind` values and the function that writes them. Payload is `meta jsonb` always containing `request_id`, `actor` (`order_flow`, `system`, `rider`), and per kind the fields below.

kind	Producer function	`meta` extras	Consumer
`delivery.created`	`dispatch_create_job`	`app 'one-local'`, `dispatch_trigger`	`jobView.timeline` (HTTP `GET /api/dispatch/jobs…`) — **no subscriber**
`delivery.released`	`dispatch_release`	`prep_minutes`	same
`delivery.offered`	`dispatch_offer_next`	`round`, `rider_id`	same
`delivery.timed_out`	`dispatch_offer_next`	`round`, `rider_id`	same
`delivery.no_rider`	`dispatch_offer_next`	`rounds`	same; also triggers a push to the restaurant (`afterOffer`)
`delivery.assigned`	`dispatch_answer`	`app 'onlatur-rider'`, `rider_id`, `round`	same
`delivery.rejected`	`dispatch_answer`	`app 'onlatur-rider'`, `rider_id`, `round`	same
`delivery.cancelled`	`dispatch_cancel`	`reason`, `rider_id`	same

Column `v int default 1` = event schema version.

POS `order_events` (OS-owned table, written by 0023 functions): `kind` values `placed` (actor `one_local`), `accepted`, `kot_fired`, `rejected`, `kot_ready`, `out_for_delivery`, `delivered`, `paid` (actor `partner`), `timed_out` (actor `one_local`). Payload `meta`: `source`, `ol_order_id`, `prep_minutes`, `by`, `kot_no`, `reason`, `method`, `auto_paused`. Consumer in this repo: `restaurant.ts getOrder` (shows the timeline in the Partner app). Other

consumers (POS UI, Master) — **UNKNOWN**.

1.3 HTTP webhooks received

EVENT	PRODUCER	CONSUMER	PAYLOAD	DB EFFECT
`payment.captured`, `order.paid`	Razorpay → `POST /api/payments/webhook`	`payments.ts handleWebhook` → `markPaid`	`payload.payment.entity {id, order_id, amount, method, notes.checkout_id}`	`ol_checkouts` → `paid` (+ trigger to `ol_orders`); `ol_payment_events` INSERT; `dispatchOrders`
`payment.failed`	Razorpay	same	`payment.entity.error_description`	`ol_checkouts.payment_status` `pending` → `failed`, `payment_error`; `ol_payment_events`
`refund.processed`	Razorpay	`recordGatewayRefund`	`payload.refund.entity {id, payment_id, amount}`	`ol_refunds` INSERT; `ol_orders` / `ol_checkouts.payment_status` → `refunded` when covered
Signed calls from another service	"VINII / operations tooling" (docs) → `/api/dispatch/*`	`dispatch.ts serviceRequest`	job payload (`docs/RIDER_DISPATCH_API.md`)	dispatch RPCs
Scheduled call	Intended: a scheduler → `POST /api/dispatch/sweep` every 15–30 s	`dispatch.ts sweep`	–	advances/cancels/creates jobs. **Who calls it: UNKNOWN** (no schedule in this repo; heartbeats and offer reads call `advanceDue` as a partial substitute)

The `.emergent/cron/*` files are the generic Emergent preview-pod cron runner (envelope `{"event":"schedule.triggered","schedule_id","run_id","dispatch_time","job_id","data":null}` POSTed with `WEBHOOK_CRON_SECRET`); `webhook-crons` contains only the watcher entry — no One Local job (**no sweep, no privacy purge job**). `docs/JOURNAL.md` (privacy entry) says the scheduled retention clean-up is not built.

1.4 Application-level events (code → code)

EVENT	PRODUCER	CONSUMER	PAYLOAD	DB EFFECT
"Order successful"	`store.ts createCheckout` (COD); `payments.ts markPaid` / `switchToCash`	`dispatch.ts dispatchOrders` → `createJobForOrder` → `createJob`	`ol_order_id`	`delivery_jobs`, `delivery_events` (see 03 §3.D)
"Restaurant accepted"	`restaurant.ts act('accept')`	`dispatch.ts onRestaurantAccepted`	`pos_order_id`, `prep_minutes`	`dispatch_release` if a job exists
"Restaurant rejected"	`restaurant.ts act('reject')`	`dispatch.ts onRestaurantRejected`	`pos_order_id`, reason	`dispatch_cancel`
"Restaurant timed out"	`ol_expire_unanswered` (inside `ol_partner_act`)	Dispatch **sweep** (`order_cancelled`)	–	`dispatch_cancel` on next sweep
Catalogue invalidation	`admin.ts` listing writes → `catalog.ts invalidateCatalog()`	catalogue cache of this process	–	none (other server processes catch up within the 30 s TTL)

1.5 Push notifications (Expo Push API `https://exp.host/--/api/v2/push/send`)

EVENT	PRODUCER	CONSUMER	PAYLOAD `data`	DB EFFECT
`new_order`	`restaurant.ts notifyNewOrder` — **called only by** `createTestOrder` (a real customer order does not push)	Partner app devices (`ol_partner_devices` of the org); sound `new_order.wav`, channel `new-orders`, ttl 60 s	`{type, order_id}`	dead tokens (`DeviceNotRegistered`) deleted
`no_rider`	`dispatch.ts afterOffer`	restaurant devices	`{type, job_id}`	same
`job_offer`	`afterOffer`	rider devices (`push_tokens`), `job_offer.wav`, channel `job-offers`, ttl 30 s	`{type, job_id, offer_id, expires_at}`	same
`job_assigned`	`afterOffer` (auto-accept)	rider	`{type, job_id}`	–
`job_cancelled`	`cancelJob`	rider	`{type, job_id}`	–

1.6 Polling and timers (substitutes for realtime)

Consumer	Mechanism	Interval	Source
Partner app: new orders	React Query `refetchInterval` on `GET /api/partner/orders?tab=new` (only while `accepting`)	5 s	`partner/src/use-alert.ts`, `use-orders.ts` (10 s for other tabs)
Partner server: timeout detection	`ol_expire_unanswered` at the start of each partner list request	per poll	`restaurant.ts listOrders`
Rider app: presence	`POST /api/rider/heartbeat`	every 30 s (rider silent > 60 s counts offline)	`rider.ts` comments
Dispatch clock	`advanceDue(requestId, minGapMs=10 s)` from heartbeats, from rider going online, from reading an expired offer	≥ 10 s	`dispatch.ts`
Customer order screen	**no polling** — one fetch on open; progress is a client timer (`TOTAL_S = 10 min`)	–	`frontend/app/order/[id].tsx`
Catalogue cache	30 s TTL; service area cache 60 s; admin cache 30 s; Supabase user lookup cache 60 s (10 s on failure); migration probes every 60 s	–	various

1.7 Log lines (no consumer in repo; for operators)

`[admin] {at, by, role, action, target, …}` — one per admin write (listing create/update/status, shopper block, media upload URL). `[privacy] {at, event, who(tag), …}`. `[payments]`, `[dispatch]`, `[push]`, `[store]`, `[db]`, `[api]` errors/warnings.

2. Events named in documents but not implemented

`checkout.created`, `order.created`, `order.accepted`, `order.cancelled`, `order.fulfilled`, `payment.*` (`docs/PLATFORM_FOUNDATIONS.md` §6.5, table `ol_order_events`); Realtime channels (D5); `pos_place_order` / `pos_mark_handed_over` POS functions (agreed names in the docs; **not in this repo, not in** `db/local/0000_os_replica.sql`; their production existence is UNKNOWN); `ol_place_checkout` single-transaction function (build-order item C5).

06 — RLS and SECURITY (One Local / OL-ui)

Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**; **DRAFT** = not applied to production per `docs/JOURNAL.md`.

0. Security model in one paragraph

Every One Local table has **Row Level Security enabled with zero policies**, and `anon` / `authenticated` have **no grants**. Only the Postgres role `service_role` can read or write. The apps never connect to the database; they call the Expo API routes, which connect with the **service-role key** (`SUPABASE_SERVICE_ROLE_KEY`, `frontend/server/db.ts`) — and the service role **bypasses RLS entirely**. So **all authorization (organization, user, location scoping) lives in application code**, not in the database. Policies are absent by design: "One Local's server (service role) can read or write these tables" (header of `0021`).

1. Per-table RLS (tables defined by this repo)

All of the following: **RLS ENABLED = yes; SELECT / INSERT / UPDATE / DELETE policies = none (no policy of any kind is created by any migration); anon = no access; authenticated = no access; service_role =** `SELECT, INSERT, UPDATE, DELETE` (granted explicitly in 0021 / 0022 / 0024 / 0025 / 0026 / 0029). Exceptions are noted.

Table	Migration	Notes
`ol_service_areas, ol_categories, ol_brands, ol_stores, ol_products, ol_banners, ol_discover_chips, ol_coupons, ol_events, ol_reels, ol_themed_stores, ol_media_assets`	0021	Public catalogue data, but **not** readable by `anon`; the customer app reads it through `GET /api/catalog` etc.
`ol_orders, ol_booking_enquiries, ol_support_requests`	0021	Contain personal data; same lock
`ol_merchant_channels, ol_partner_devices`	0022	
`ol_order_items, ol_refunds`	0024	
`ol_payment_events`	0025	
`ol_checkouts`	0026 DRAFT	
`ol_consents`	0027 DRAFT	grant is `select, insert, delete` only (no `update`) — "delete only passes the purge guard" trigger
`ol_privacy_requests`	0027 DRAFT	`select, insert, update, delete`
`ol_consent_current` (view)	0027 DRAFT	`security_invoker = true`; `revoke all` from anon/authenticated; `grant select` to service_role
`riders, rider_store_links, rider_availability, push_tokens, org_dispatch_settings, delivery_jobs, delivery_assignments, delivery_events`	0029 DRAFT	`delivery_events_id_seq`: `usage, select` to service_role
`delivery_orders_needing_rider` (view)	0029 DRAFT	`revoke all` from anon/authenticated; `grant select` to service_role. Created without `security_invoker` (view owner semantics) — runs with the privileges of the view owner; access is controlled only by the revoke

Functions: every `ol_*` and `dispatch_*` function: `revoke all … from public, anon, authenticated; grant execute … to service_role`. They are `SECURITY INVOKER` (not definer), so they run as the caller (service role).

Storage: bucket `onecity-catalog` is `public = true` (anyone can read objects by URL). Storage policies for writes: **none defined in this repo** (UNKNOWN what Supabase's `storage.objects` policies are in production); writes in the app happen only through service-role–generated signed upload URLs.

2. OS / POS tables (owned elsewhere) — what is verifiable

Table	RLS	Policies
`organizations, roles, org_users, org_counters, menu_categories, menu_items, item_channel_status, channel_integrations, orders, order_items, order_events, kot_tickets, payments, schema_migrations`	`db/local/0000_os_replica.sql` enables RLS and grants all to service_role **on the local replica only**	**UNKNOWN in this repo.** The replica has no `CREATE POLICY`. `docs/CENTRAL_DB_MAP.md` (production snapshot 2026-09-22) reports policy **counts** only: organizations 5, org_users 5, menu_items 5, orders 5, order_items 5, kot_tickets 5, order_events 4, payments 4, item_channel_status 4, channel_integrations 4, platform_admins 1, `locations` 1, `menu_item_variants` 4; 140 policies total; RLS on for all except `schema_migrations`. Conditions are not recorded
`platform_admins`, `locations`, `menu_item_variants`	not in the replica	UNKNOWN

Journal warnings relevant to POS tables (not verifiable here): "The Master's web POS can change a kitchen ticket without checking which restaurant it belongs to" and "The Windows POS has a draft table open to anyone" (`docs/JOURNAL.md`, 2026-09-28) — both outside this repo.

The first-draft `ol_*` tables in the 2026-09-22 snapshot had 1 policy each (`ol_banners` etc.); 0021 drops and recreates them without policies.

3. Application-level authorization (what actually scopes data)

Layer	Code	Rule
Customer identity	`auth.ts caller()`	`owner = sha256("user:"+userId)` if a valid Supabase JWT (`GET {SUPABASE_URL}/auth/v1/user`, result cached 60 s; failures 10 s) else `sha256(x-customer-key)`. `matches` = [user hash, device hash]. All order reads filter `customer_hash IN matches`
Customer write	`store.ts requireCustomer`	requires `owner` (device key **or** login). The server does **not** require sign-in for checkout; the app does
Customer privacy routes	`privacy.ts person()`	login required (`401` otherwise); limit per account tag
Partner (restaurant)	`partner.ts requirePartnerAuth`	JWT → `org_users` row with `status='active'` (earliest membership if several); org id from `organizations`; roles → actions map `CAN` (`accept/reject/outlet/items/ready/out/delivered`). All Partner queries filter `organization_id = staff.organizationId`; `act()` re-reads `orders.organization_id` before calling the RPC. **The RPC `ol_partner_act` itself does not check organization** (it is service-role only; the check is in `restaurant.ts`)
Rider	`rider.ts requireRider`	JWT → `riders` row with `status='approved'`; `suspended` /other statuses refused. Rider reads `delivery_jobs` filtered by `rider_id = me`; offer answer function checks `p_rider` owns the assignment
Platform admin	`admin.ts requireAdmin`	JWT → `platform_admins` row (cached 30 s); write roles `master_admin, onboarding_admin, operations`; 60 writes / 240 reads per minute per admin; origin allow-list (`MASTER_APP_URL`, localhost outside production); dev tokens `dev-admin:<role>` only without a database and never in production
Dispatch service	`dispatch.ts serviceRequest`	bearer `DISPATCH_SERVICE_TOKEN` (constant-time compare via HMAC) + `x-timestamp` within ±5 min + `x-signature` = HMAC-SHA256(`DISPATCH_SIGNING_SECRET`, `timestamp.rawBody`) + `x-request-id` + `Idempotency-Key` on writes; optional `DISPATCH_ALLOWED_ORGS` (`*` = also stores without a merchant). A user's JWT is not accepted
Razorpay	`payments.ts`	webhook HMAC with `RAZORPAY_WEBHOOK_SECRET`; verify HMAC with `RAZORPAY_KEY_SECRET`; amounts recomputed server-side; the checkout page requires the owner's headers
Rider privacy	`dispatch.ts riderJob`	address, phone and exact drop point revealed only by status (area ~1 km rounded before pickup; customer phone only while job live; delivery address after `picked_up`)
Partner privacy	`restaurant.ts shape`	customer first name only; phone and full address only while status in `accepted, packed, out`

4. Secrets and environment (names only; no values read)

Server: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_PUBLISHABLE_KEY` / `SUPABASE_ANON_KEY`, `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`, `DISPATCH_SERVICE_TOKEN`, `DISPATCH_SIGNING_SECRET`, `DISPATCH_OTP_SECRET`, `DISPATCH_ALLOWED_ORGS`, `GOOGLE_MAPS_API_KEY`, `MASTER_APP_URL`, `PRIVACY_HARD_DELETE`, `NEARBY_SOURCE`, `FEATURE_<NAME>`, `ANTHROPIC_API_KEY/OPENAI_API_KEY/EMERGENT_LLM_KEY` (flag only). Client-shipped (`app.config.ts extra`): `backendUrl`, `supabaseUrl`, `supabaseAnonKey` (publishable), `googleMapsBrowserKey`. `scripts/db.py` reads `SUPABASE_ACCESS_TOKEN` (Management API, read-only commands). `.env` is git-ignored; `frontend/.env.local` / `partner/.env.local` point development at the local Supabase.

5. Security observations (verified facts, not fixes)

1. **Service role everywhere.** One key can read/write every table; a bug in an application-level filter is a data leak with no database backstop.

2. **Checkout without login is accepted by the server** (device key only). Orders so created have `user_id NULL`; delivery job creation then has no customer name/phone (`createJobForOrder` reads them only when `o.user_id`).
3. **Banned/blocked users:** `blockShopper` sets a Supabase Auth ban. The server accepts an access token that Supabase already validated for up to 60 s (cache in `auth.ts`); refresh stops. (INFERRED duration of exposure.)
4. **Rate limits are per process** (`http.ts limit`), not shared; `limit()` buckets reset on restart.
5. **Admin dev tokens** (`dev-admin:*`) exist in code; guarded by `dbConfigured()` false and `NODE_ENV !== 'production'`.
6. `ol_partner_act` trusts its caller for organization scoping (service role only); the same for `ol_process_refund` (no actor/role check in SQL; also no caller).
7. `ol_orders.delivery_address` (name/phone not included; text + lat/lng) is returned to the owner and to riders after pickup; erasure nulls it.
8. **Public bucket** `onecity-catalog` (catalogue and store images only).
9. `ol_consents.user_id`, `ol_privacy_requests.user_id` have **no FK** by design, so evidence survives deleting the login.
10. The partner `PATCH /api/partner/outlet` reads `await request.json()` directly (not the size-limited `readBody`), unlike the other routes.
11. `.emergent/cron/webhook_crond.sh` reads `WEBHOOK_CRON_SECRET` from `/app/backend/.env` (deployment-pod mechanism; separate from the app's own secrets).

07 — OFFLINE BEHAVIOUR and SYNC (One Local / OL-ui)

Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**.

0. Verdict

This application does not support offline operation. Searches of `frontend/`, `partner/`, `frontend/server/` for `sqlite`, `expo-sqlite`, `realm`, `watermelon`, `outbox`, queue/replay of mutations found no local database, no mutation queue, no sync engine, no last-sync timestamp, no conflict resolution. Not invented below.

Item asked for	Answer
Local database	**None.** Only key/value storage: `@react-native-async-storage/async-storage` (and `frontend/src/utils/storage/index.ts` also wraps `expo-secure-store` (Keychain) for native "secure" values; which screens call it was not traced — **UNKNOWN**)
SQLite tables	None
Local IDs	Device customer key (random, below); cart line ids are catalogue ids; saved address ids (`Address.id`) are generated on the phone. Orders get their id on the **server** (`crypto.randomUUID()` inside `createCheckout`) — no client-generated order id
Server IDs	Same as above
Sync mechanism	None. Reads are on-demand REST + React Query caching (`frontend/src/query-client.ts` is a default `QueryClient`; catalogue/feature queries set `staleTime` 60 s; `partner/src/query-client.ts` sets `retry 1`, `staleTime` 5 s)
Outbox	None on the phones. (A server-side `delivery_events` outbox exists for dispatch — not an offline mechanism)
Retry	Customer: manual retry buttons; cart screen re-sends the same `Idempotency-Key` for the same basket (`checkoutKey` ref) so a retry/double tap returns the same checkout. After sign-in the cart "carries on paying by itself" (in-memory `payAfterSignIn`). Consents recorded while signed out are kept on the phone and re-sent after sign-in (`frontend/src/privacy.ts`). Partner/rider: polling re-fetches
Idempotency	Server-side only: `Idempotency-Key` → unique `(customer_hash, idempotency_key)` on `ol_checkouts` (or `ol_orders` pre-0026); dispatch `delivery_jobs.idempotency_key = 'onelocal:<ol_order_id>:delivery'` and `ol_order_id` UNIQUE; Razorpay events deduped on `ol_payment_events.event_id`; refunds on `ol_refunds.reference` unique; `ol_partner_act` uses an optimistic version check (`orders.updated_at`) → `OL_STALE`
Conflict resolution	Optimistic concurrency only for partner actions (`version` = `orders.updated_at`; mismatch → 409 "already changed on another phone"); dispatch uses `delivery_jobs.version` and partial unique indexes (first accept wins)
Last-sync timestamp	None
Queued mutations	None
Duplicate prevention	See Idempotency; plus first-wins constraints on offers (`delivery_one_live_offer_per_job/rider`, `delivery_one_accepted_per_job`)
Offline UX	`frontend/src/components/error-boundary.tsx` and per-screen error text with "Try again"; the cart is persisted locally so it survives being offline

1. What is persisted on the phone (AsyncStorage keys found)

Key	File	Content	Notes
`onecity.customer-key.v1`	`frontend/src/api.ts`	random 48-hex device key sent as `x-customer-key`	removed by "Clear this phone"; the next request makes a new one
`onelatur.cart.v2` / `onelatur.cart.v1`	`frontend/src/cart.tsx`	`{food:[...], shopping:[...], kind}` (v1 = single list, migrated)	the **only** cart — no server cart

Key	File	Content	Notes
`onecity.customer.v1`	`frontend/src/customer.tsx`	profile (name, email, phone, avatar, photo data URI, birthday, diet, language, notification prefs), **saved addresses**, last payment method/UPI app, referral code	the address book exists only here
`onecity.saved-products.v1`	`customer.tsx`	wishlist product ids	
`onecity.privacy.v1`	`frontend/src/privacy.ts`	consent choices + notice version (re-sent to server after sign-in)	
`onelatur.booking-enquiries`	`frontend/src/bookings.ts`	copies of submitted Book It enquiries	local list of server-saved rows
`onelatur.followed.v1`	`frontend/src/followed.ts`	followed store ids	
`onecity.recent-places.v1`	`frontend/src/places.ts`	last 5 searched places	
`onecity.location-asked.v1`	`app-shell.tsx`	flag	
`one-latur-location`	`frontend/app/addresses.tsx`	chosen delivery location text	
`onecity-demo-wallet`	`frontend/app/wallet.tsx`	demo wallet entries	not connected to any server balance
`onecity-ai-session`	`frontend/app/assistant.tsx`	AI chat session id	assistant API is switched off
`reel-like-<id>`	`reel-card.tsx`	like flag	
launch-request flag `LAUNCH_KEY(place.id)`	`explore-town.tsx`	"I asked for this town"	
Supabase session	`frontend/src/lib/supabase.ts`, `partner/src/lib/supabase.ts`	auth session persisted in AsyncStorage (`persistSession`, PKCE) on native; browser storage on web	
`partner-alert-seen`	`partner/src/use-alert.ts`	declared constant; the seen set is an **in-memory** `Set` (not persisted)	

`frontend/src/privacy.ts` lists every key the app writes (for the DPDP "what is on this phone" export and "Clear this phone").

2. Server-side "no database" mode (not offline sync)

When `SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY` are absent, the server answers from memory (`store.ts`, `privacy.ts`, `marketplace.ts`) and `frontend/server/data/catalog.json`. This is a **development fallback** for UI work (`expo start` with no `.env`); data is lost on restart.

3. Offline-relevant gaps (facts)

- A customer who loses connectivity mid-payment: Razorpay's webhook is the source of truth; the app re-checks `GET /api/checkouts/:id` after the WebView closes (`cart.tsx onPaymentResult`).
- The Partner app relies on 5 s polling and push; with no connection it shows nothing new. Orders time out server-side after the accept window (default 60 s) regardless.
- The Rider app is not in this repo; its offline behaviour is **UNKNOWN**.

08 — EXTERNAL INTEGRATIONS (One Local / OL-ui)

Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**; **DRAFT**. Direction: OUT = this app calls it; IN = it calls this app; BOTH.

INTEGRATION | DIRECTION | ENDPOINT/SERVICE | DATA SENT | DATA RECEIVED | TRIGGER | AUTHENTICATION

INTEGRATION	DIRECTION	ENDPOINT / SERVICE	DATA SENT	DATA RECEIVED	TRIGGER	AUTHENTICATION
Supabase Postgres (PostgREST + RPC) — project `ywylhzcxuoeilygzdonx` is production for every Proviyaa app (`docs/PLATFORM_FOUNDATIONS.md` D15, `CENTRAL_DB_MAP.md`); local dev project `onelocal-dev`	OUT (server only)	`${SUPABASE_URL}/rest/v1/*` via `@supabase/supabase-js` in `frontend/server/db.ts`	all reads/writes in `04_API_MAP.md`	rows / RPC results	every API request	`SUPABASE_SERVICE_ROLE_KEY` (service role; bypasses RLS)
Supabase Auth (customers, staff, riders, admins)	OUT (client + server)	client: `supabase.auth.signUp / signInWithPassword / signInWithOtp+verifyOtp (phone SMS) / signInWithOAuth(google, PKCE) / getSession / onAuthStateChange` (`frontend/src/auth.tsx`, `partner/src/auth.tsx`); server: `GET /auth/v1/user` (token check, `auth.ts`, `privacy.ts accountOf`), `POST /auth/v1/logout?scope=global` (`privacy.ts endLogin`), `auth.admin.getUserById / listUsers / updateUserById(ban_duration) / deleteUser` (`dispatch.ts`, `admin.ts`, `privacy.ts`); `GET /auth/v1/settings` (`auth.tsx`, which sign-in methods are on)	email, password, phone, OTP, user JWT	session, user record (id, email, phone, metadata `full_name`, `providers`)	sign-in screens; every authenticated API call	client: publishable/anon key (`SUPABASE_PUBLISHABLE_KEY` / `SUPABASE_ANON_KEY` shipped in `extra`); server admin calls: service-role key
Supabase Storage	OUT	bucket `onecity-catalog` (public) — signed upload URL created by `admin.ts uploadUrl` (`createSignedUploadUrl`), the Master PUTs the file straight to Storage; catalogue media served by 302 from `/api/media/<key>` using `data/media.json`	image file (≤ 5 MB checked in code; bucket limit 40 MiB), path `stores/YYYY/MM/<hex>-<slug>.<ext>`	public URL	Master listing editor	service role (to mint the signed URL); upload itself uses the signed URL
Razorpay (payments)	BOTH	OUT: `POST https://api.razorpay.com/v1/orders`, `GET /v1/payments/{id}` (`payments.ts`). Client: `https://checkout.razorpay.com/v1/checkout.js` inside a React Native WebView page served by `/api/payments/checkout-page`. IN: `POST /api/payments/webhook`	order amount (paise, from the server total), `receipt = checkout id`, `notes.checkout_id`, customer prefill name/email/contact (handed to the page by the app after load, not in a URL)	Razorpay order id, payment entity (id, method, amount, status), events `payment.captured`, `order.paid`, `payment.failed`, `refund.processed`	online payment started / verify / Razorpay dashboard refund	OUT: Basic auth `RAZORPAY_KEY_ID:RAZORPAY_KEY_SECRET`. IN: HMAC-SHA256 `RAZORPAY_WEBHOOK_SECRET` on raw body (`x-razorpay-signature`); verify: HMAC with key secret. Card/UPI details never reach this server. No refund API call is made by this repo (refunds are recorded after Razorpay reports them)
Expo Push	OUT	`POST https://exp.host/--/api/v2/push/send` (`push.ts`)	token list, title, body, `data`, sound, `channelId`, ttl (60 s restaurants, 30 s riders)	tickets (dead tokens removed)	`new_order` (test-order only), `no_rider`, `job_offer`, `job_assigned`, `job_cancelled`	none (Expo push needs no server secret per `push.ts` header comment)
Google Maps Platform	OUT	Places API (New): `places.googleapis.com/v1/places:autocomplete`, `/places/{id}`, `places:searchNearby`; Geocoding `maps.googleapis.com/maps/api/geocode/json` (`maps.ts`, `nearby.ts`); client map rendering via `react-native-maps` / `@vis.gl/react-google-maps` using `googleMapsBrowserKey`	query text, session token, lat/lng, India bias	place predictions, address components, nearby places	address search, pin drop, discover-town page	`GOOGLE_MAPS_API_KEY` (server); `GOOGLE_MAPS_BROWSER_KEY` (web client). Feature `maps` on only when the key exists
OpenStreetMap Overpass	OUT	`overpass-api.de`, `overpass.private.coffee` (`nearby.ts`)	bounding query by area	restaurants/groceries/ shops/pharmacies	"discover" towns when no	none

INTEGRATION	DIRECTION	ENDPOINT / SERVICE	DATA SENT	DATA RECEIVED	TRIGGER	AUTHENTICATION
				(ODbL; app shows credit)	Google key (or `NEARBY_SOURCE=osm`)	
Expo / EAS Updates (OTA)	IN to the phone	`https://u.expo.dev/<EAS_PROJECT_ID>` (`app.config.ts`)	JS bundle	–	app launch (`checkAutomatically: 'ON_LOAD'`) when `EAS_PROJECT_ID` is set	EAS project
Apple Live Activities / WidgetKit	local	`frontend/modules/onecity-live-activity`, `frontend/targets/widget` (app group `group.<bundle id>`)	order stage, ETA, item count, total (from the **client timer**)	–	order screen	on-device
AI providers (Anthropic / OpenAI / Emergent LLM)	–	env keys only toggle `features().ai_search/assistant`; `/api/assistant/*` and `/api/search/*` always answer 503 (switched off while moving to the new backend)	–	–	–	–
Emergent object storage / FastAPI backend (legacy)	OUT (legacy)	`backend/media_store.py` → `integrations.emergentagent.com/objstore/api/v1/storage` with `EMERGENT_LLM_KEY`; no server entry file for the FastAPI app is in the repo	–	–	–	**INFERRED legacy**; replaced by Expo API routes + Supabase Storage
Emergent webhook cron	IN (pod-local)	`.emergent/cron/*` runs `dispatch_webhook.sh` posting `{"event":"schedule.triggered",…}` to a configured endpoint with `WEBHOOK_CRON_SECRET`; the repo's `webhook-crons` file defines only the watcher	–	–	per-minute watcher	`WEBHOOK_CRON_SECRET` from `/app/backend/.env`. **No One Local job is scheduled here**
Master admin (Proviyaa OS console, repo `loki1514/Repo-1-`)	IN	`/api/admin/*` (CORS allow-list `MASTER_APP_URL`)	listing JSON, shopper block, upload requests	JSON lists	Master operator actions	admin's own Supabase JWT + `platform_admins` row
VINII POS / OS (Flutter + web)	via shared DB	POS tables `orders`, `order_items`, `kot_tickets`, `payments`, `order_events`, `menu_*`, `item_channel_status`	`ol_create_restaurant_order`, `ol_partner_act`	–	restaurant orders (P2 path only)	service role through RPC. No HTTP call between apps. POS functions named in the docs (`pos_place_order`, `pos_mark_handed_over`) are **not called** by this repo
One Local Delivery / Rider app (separate repo "One Local Delivery app")	IN	`/api/rider/*` (rider JWT) and `/api/dispatch/*` (service-signed)	availability, heartbeat, push token, accept/decline	offers, jobs	rider actions	rider JWT / service token + HMAC
Partner (VINII Partner) app — in this repo (`partner/`)	IN	`/api/partner/*`	see 04	–	staff actions	staff JWT
Printers / hardware / thermal / Bluetooth	–	**None in this repo** (searched). The KOT is only a database row (`kot_tickets`)	–	–	–	–
WhatsApp	–	**None.** (`whatsapp` appears only as a UPI app choice in `payment-options.tsx` and in the POS `channel_integrations` channel list)	–	–	–	–
SMS	via Supabase	phone OTP is sent by Supabase Auth's configured provider (provider **UNKNOWN**; `supabase/config.toml` local defaults)	–	–	–	–
Email	via Supabase	sign-up confirmation by Supabase Auth (local config uses Inbucket/`local_smtp`)	–	–	–	–

INTEGRATION	DIRECTION	ENDPOINT / SERVICE	DATA SENT	DATA RECEIVED	TRIGGER	AUTHENTICATION
Delivery services (third party)	–	none besides One Local's own rider pool	–	–	–	–
Analytics / crash reporting SDKs	–	none found in `package.json`	–	–	–	–
Higgsfield (character generation)	–	`frontend/src/components/higgsfield-character.tsx` contains only a commented-out `fetch('/api/higgsfield/generate')`	–	–	–	–
Playwright / test reports	dev	`frontend/e2e/*`, `test_reports/*`, `tests/` (Python pytest XML results of the old backend)	–	–	–	test tooling only

09 — ROUTES AND SCREENS (One Local / OL-ui)

Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**. Format: SCREEN/ROUTE → USER ACTION → API/RPC → DATABASE TABLES → SIDE EFFECTS. Screens are Expo Router files. Tab bar (`frontend/app/(tabs)/_layout.tsx`): `index, categories, discover, food, book-it` visible; `reels, cart, account, grocery, pharmacy, beauty, shops, care` hidden tabs (reachable by link). "Catalogue" = `GET /api/catalog` (+ `/products`, `/stores`) → `ol_*` catalogue tables and item master (see 04). Bundle names: customer app display name "OneCity" (`app.config.ts`), repo/branding also "One Latur", "One Local".

1. Customer app (`frontend/app/`)

SCREEN / ROUTE	USER ACTION	API / RPC	DB TABLES	SIDE EFFECTS
`(tabs)/index` (Home)	open	`GET /api/stores` (`api<Store[]>('/stores')`), `GET /api/catalog` via `useCatalog`	`ol_stores, ol_products, ol_categories, ol_brands, ol_banners, ol_coupons, …` (cached)	–
`(tabs)/categories`, `category/[id]`, `collection/[id]`, `brand/[id]`	browse	`GET /api/catalog`	catalogue	–
`(tabs)/food`, `grocery`, `shops`, `pharmacy`, `care`, `beauty` (department screens)	browse a department	`GET /api/catalog` (department filter client-side)	catalogue	–
`store/[id]`	open a store	`GET /api/stores/:id`, `/stores/:id/products`, `/categories`	`ol_stores` + products	–
`product/[id]`	open product; add to cart	`GET /api/products/:id/story`	catalogue	cart (AsyncStorage `onelatur.cart.v2`)
`search`, `smart-search`	text search; "discover" places	`GET /api/products?q=`; `GET /api/nearby`	catalogue; Google Places or Overpass	photo/voice AI search is switched off (503)
`(tabs)/discover`, `(tabs)/reels`	browse chips and reels	`GET /api/reels`, `/catalog`, `/discover`	`ol_reels, ol_discover_chips, ol_products`	like flag stored locally
`(tabs)/book-it`, `booking/[id]`	choose event, date, slot, guests, name, phone; submit	`GET /api/events`; `POST /api/booking-enquiries`	`ol_events`, INSERT `ol_booking_enquiries`	copy saved to AsyncStorage `onelatur.booking-enquiries`; no merchant notification
`enquiries`	view past enquiries	none (reads local copy)	–	–
`location`, `india`, `address-map`, `address-form`, `addresses`	choose delivery town/address	`GET /api/coverage`, `/india/*`, `/places/*`, `/geocode/reverse`, `/serviceability`; `POST /api/launch-requests` (discover-only town)	`ol_service_areas` (coverage); INSERT `ol_support_requests` (`launch:<id>`)	saved addresses live in AsyncStorage `onecity.customer.v1`; chosen location in `one-latur-location`
`(tabs)/cart`	open	`POST /api/cart/quote` (re-quoted on change); `POST /api/coupons/apply`; GET `/api/features`	reads catalogue/coupons	–
`(tabs)/cart`	**Pay on delivery**	`POST /api/checkouts` (`Idempotency-Key`)	INSERT `ol_checkouts`, `ol_orders`, `ol_order_items`	`dispatchOrders` → `delivery_jobs`, `delivery_events`, rider push; navigates to `order/[id]`
`(tabs)/cart`	**Pay online** (when `online_payments`)	`POST /api/payments/order` → Razorpay Checkout in WebView (`/api/payments/checkout-page`) → `POST /api/payments/verify`; fallback `GET /api/checkouts/:id`; `POST /api/payments/cod` to switch	`ol_checkouts` (INSERT, UPDATE), `ol_orders` (INSERT + trigger updates)	Razorpay order; on paid: `dispatchOrders`
`sign-in`	email+password / sign-up / phone OTP / Google	Supabase Auth direct	`auth.users`	consent recorded (`privacy.ts recordConsent`), then `POST /api/privacy/consents` after sign-in
`orders` (My Orders) / `(tabs)/account`	open	`GET /api/orders`	`ol_orders`	–
`order/[id]` (tracking)	open an order	`GET /api/orders/:id`, `GET /api/checkouts/:id` (sibling store orders)	`ol_orders`, `ol_checkouts`	**progress is a local timer** (10 min); iOS Live Activity + widget updated from it; rider shown as "Sample rider"

SCREEN / ROUTE	USER ACTION	API / RPC	DB TABLES	SIDE EFFECTS
`help`	send a question	`POST /api/support-requests`	INSERT `ol_support_requests`	–
`privacy`, `privacy-notice`	export data / manage consents / delete account / grievance / nominee / "Clear this phone"	`GET /api/privacy/export`, `GET|POST /api/privacy/consents`, `POST /api/privacy/erasure`, `POST /api/privacy/grievances`, `POST /api/privacy/nominee`	`ol_orders, ol_checkouts, ol_refunds, ol_support_requests, ol_booking_enquiries, ol_consents, ol_privacy_requests`	erasure signs out everywhere; AsyncStorage wiped
`profile-edit`, `payments` (preferred method), `wishlist`, `refer`, `about`, `wallet`	local settings	none	none	AsyncStorage only (`onecity.customer.v1`, `onecity.saved-products.v1`, `onecity-demo-wallet`); referral rewards and wallet are not active
`assistant`	chat	`/api/assistant/*` → 503	–	switched off
`+html.tsx`, `_layout.tsx`	shell	–	–	`_layout` wraps Auth, Cart, Customer providers and the React Query client

2. VINII Partner app (`partner/app/`)

SCREEN	USER ACTION	API / RPC	DB TABLES	SIDE EFFECTS
`sign-in`	email + password	Supabase Auth direct, then `GET /api/partner/me`	`auth.users`, `org_users`, `roles`, `organizations`, `ol_merchant_channels`	403 if not active staff of an organization
`index`	open	redirect	–	–
`orders`	tabs New / Preparing / Ready / Past; pulls every 5 s (new) / 10 s	`GET /api/partner/orders?tab=`	RPC `ol_expire_unanswered`; `ol_orders` ⋈ `orders` ⋈ `order_items`	unanswered orders get cancelled by the RPC
`alert` (full-screen incoming order)	polled new order auto-opens; **Accept** (pick prep time) / **Reject** (reason)	`POST /api/partner/orders/:id/accept|reject`	RPC `ol_partner_act` (`orders, order_items, kot_tickets, order_events, ol_orders, ol_merchant_channels`)	accept → KOT row + `dispatch` release; reject → `dispatch_cancel`
`order/[id]`	view; **Ready**, **Out for delivery**, **Delivered**; call customer (`tel:`)	`GET /api/partner/orders/:id`, `POST …/ready|out|delivered`	RPC `ol_partner_act`; `order_events` read	delivered writes POS `payments` (cash)
`items`	switch an item on/off for One Local	`GET /api/partner/items`, `PATCH /api/partner/items/:id`	`menu_categories, menu_items, item_channel_status`	item hidden/visible on the customer app within the 30 s catalogue cache
`settings`	Accepting / not accepting; register push device; **Send test order**	`PATCH /api/partner/outlet`, `PUT /api/partner/devices`, `POST /api/partner/test-order`	`ol_merchant_channels`, `ol_partner_devices`; test order → RPC `ol_create_restaurant_order`	test order pushes "New order…"

3. Master admin (not in this repo) calling `/api/admin/*`

ROUTE	ACTION	API	TABLES	SIDE EFFECTS
Organizations list	search	`GET /api/admin/organizations`	`organizations, locations, menu_*, item_channel_status, ol_stores`	–
Listing create / edit / preview / approve / pause	operator	`POST|GET|PATCH /api/admin/listings…`, `POST …/status`	`ol_stores`, `ol_merchant_channels`	catalogue cache invalidated; audit log line
Upload logo / cover	operator	`POST /api/admin/media/upload-url`	Storage `onecity-catalog`	–
Shoppers	list / block	`GET /api/admin/shoppers`, `POST …/:id/block`	`auth.users` (admin API), `ol_orders`	Auth ban
Demand	view	`GET /api/admin/demand`	`ol_support_requests`	–

4. Rider and dispatch (callers outside this repo)

ROUTE	CALLER	TABLES
`/api/rider/*`	One Rider app	`riders, rider_availability, push_tokens, delivery_assignments, delivery_jobs`
`/api/dispatch/*`	VINII / operations tooling (signed)	`delivery_*`, `ol_orders`

5. Gaps visible from the screen map

- No screen or API to **cancel an order**, **request a refund**, **view real delivery status**, **view the delivery code**, or **rate**.
- The customer has no way to see that a **restaurant never saw** their food order (see `03_ORDER_LIFECYCLE.md` §0).
- Wallet, Coins, referrals are placeholders.

10 — DUPLICATES AND CONFLICTS (One Local / OL-ui)

Reported only. Nothing here is a fix or a recommendation. Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**; **DRAFT**.

1. Duplicate tables / records for the same business entity

#	Entity	Where it lives	Conflict
1	**Restaurant order**	`ol_orders` (customer side) **and** POS `orders` (kitchen side), linked only by `ol_orders.pos_order_id` / `orders.external_ref = 'onelocal:<ol id>'`	Intentional "one order, two sides" (`PLATFORM_FOUNDATIONS.md` D17). But the link exists only when the order is created by `ol_create_restaurant_order`; the real customer checkout writes `ol_orders` alone (see #2)
2	**Same order, two creation paths**	`store.ts createCheckout` (customer; no org, no POS link) vs `ol_create_restaurant_order` (only the Partner test order calls it)	Different pricing code (TypeScript `pricing.ts` vs SQL using `menu_items.price`, ignoring variants, GST fixed 5%), different ids (`<store>_<menu item>_<variant>` text vs `menu_item_id` uuid), different payment (`not_charged/cod|online` vs hard-coded `cod`), different line shape. The SQL function produces `ol_orders.items[].id = 'mi_<uuid>'`, the checkout produces catalogue ids
3	**Order lines**	`ol_orders.items` (jsonb), `ol_order_items`, POS `order_items`	jsonb is declared "a read cache, not truth" (`0024` comment) but it is what all reads use; `ol_order_items` is written best-effort after commit (INFERRED to fail for marketplace product ids: FK `product_id → ol_products(id)`; `ol_products` is empty after the demo clean-up; `menu_item_id` is never filled by `store.ts`)
4	**Payment state**	`ol_checkouts` (0026 DRAFT) **and** `ol_orders` (same 4 fields + Razorpay ids in 0025)	Duplicated by design: a trigger copies checkout → orders. If both 0025 and 0026 apply, the four `ol_orders` Razorpay columns stay empty (0026 header). Before 0026, `ol_orders` is the only copy
5	**Payment record**	Razorpay (money), `ol_checkouts` (our record), POS `payments` (cash only, on staff "delivered"), `ol_payment_events` (webhook log)	No single payments ledger. `ol_orders.payment_method='cod'` vs POS `orders.payment_method='cash'` / `payments.method='cash'` for the same cash event
6	**Product / catalogue**	`ol_products` (+ `ol_categories`, `ol_brands`) vs POS `menu_items` + `menu_item_variants` + `menu_categories` projected at read time (`marketplace.ts listingCatalog`)	Two live sources merged in `catalog.ts withListings` (real listings first, demo second, "hidden" set removes demo rows whose store is a listing). Product id formats differ (`p4` vs `<store>_<uuid>[_<uuid>]`). `ol_products.menu_item_id` links them but is nullable and unused by the projection
7	**Store / merchant**	`ol_stores` (text id) vs `organizations` (uuid) vs `locations` (uuid) vs `ol_merchant_channels` (org-keyed)	One physical restaurant is: an organization, optionally a location, an `ol_stores` listing (`organization_id`, `location_id`), and an `ol_merchant_channels` row. Pickup address/lat/lng are stored in **both** `ol_stores.lat/lng` (0028) and `ol_merchant_channels.address/lat/lng` (`admin.ts saveChannel` writes both); dispatch prefers `ol_stores` then the channel row
8	**Channel / accepting switch**	`ol_merchant_channels.accepting` (One Local) vs POS `channel_integrations.is_accepting` (Swiggy/Zomato/ONDC/website/WhatsApp/QR) vs `ol_stores.status` (`paused`) vs `organizations.status`	`channel_integrations.channel` CHECK does not include `'one_local'`, so One Local did not reuse it. `item_channel_status.channel` does use `'one_local'`
9	**Customer**	`auth.users` (+ `user_id`), `customer_hash` (sha256 of `user:<id>` or of the device key), OS `customers` (CRM, unrelated), planned-only `shoppers`, POS `orders.customer_name/customer_phone` (free text)	There is **no customer table**. The same person has two hashes (device before sign-in, account after) and `matches` merges them at read time. `delivery_jobs.customer_first_name/phone` is a third copy (snapshot from Auth). Erasure replaces the hash with `erased:<uuid>`
10	**Push token**	`ol_partner_devices` (restaurant staff) vs `push_tokens` (riders, 0029)	Two tables for the same Expo token concept; different columns (`organization_id,user_id` vs `rider_id`), platform CHECK differs (`ios,android,web` vs `ios,android`)
11	**Riders**	`riders` (+ `rider_store_links`, `rider_availability`) vs `ol_orders.rider_name/rider_phone/eta_minutes`	Order carries placeholder rider fields (`'Sample rider'`, `''`) that nothing updates from the real job
12	**Service area / city**	DB `ol_service_areas` (read by `coverage.ts`, `admin.ts requireCity`) vs hard-coded `ACTIVE_CITIES` and `SERVICE_AREAS` in `catalog.ts` (used by `resolveCity`, `serviceability`, dispatch drop validation, rider "go online", `maps.ts` bias)	Adding a city row in the DB does not enable it for ordering, dispatch or rider checks. Comment in `coverage.ts` says "opening a city is data, not an app release" — true only for the coverage screen
13	**Delivery fee rule**	`pricing.ts PRICING` (₹15 below ₹199 per store) vs `catalog.ts deliveryFee()` (same numbers, unused) vs the SQL function which takes `delivery_fee` from its caller	Three places for one rule; only `pricing.ts` is enforced on checkout
14	**GST rule**	`pricing.ts` (5% on food added; packaged goods inclusive, sample rates) vs `ol_create_restaurant_order` (`v_gst_pct := 5`, `orders.gst_pct`) vs POS `org_bill_settings` (sgst/cgst/service charge per org; **not read here**)	Same number, independent implementations; per-organization bill settings in the OS are ignored

#	Entity	Where it lives	Conflict
15	**Delivery address**	`ol_orders.delivery_address` jsonb `{label,text,lat,lng,notes}`; saved addresses in the phone `Address {id,label,name,phone,line,area,pin,kind,lat,lng,building,street,locality,formatted,instructions}`; `delivery_jobs.drop_*` columns; Partner reads `address.area ?? address.locality`	Two address shapes; the server shape drops `name`, `phone`, `pin`, `building`. The Partner test order passes a third shape `{label,line,area}`
16	**Help-request table reused**	`ol_support_requests` carries support questions, "launch here" demand (`reference 'launch:<place>'`) and (pre-0027) privacy requests (`reference 'privacy:<ref>'`)	One table, three meanings, distinguished by a text prefix
17	**Event naming**	`ol_events` = Book-It listings (movies/events/activities); `order_events` (POS) and `delivery_events` = event logs; `ol_payment_events` = webhook log	"events" means three different things
18	**Order tracking truth**	`ol_orders.status`, `orders.status` (POS), `delivery_jobs.status`, client timer `stageAt(elapsed)`	Four sources of progress; they are not synchronised (see §2)

2. Conflicting status fields

Field	Values	Written by
`ol_orders.status`	`placed, accepted, packed, out, delivered, cancelled`	`ol_partner_act` (only); checkout inserts `placed`
POS `orders.status`	`new, in_billing, sent_to_kitchen, ready, awaiting_payment, paid, delivered, cancelled`	`ol_partner_act`
POS `kot_tickets.status`	`new, preparing, ready, delivered`	`ol_partner_act`
POS `order_items.status`	`pending, preparing, ready, delivered, cancelled`	`ol_partner_act`
`delivery_jobs.status`	`created, offered, assigned, at_pickup, picked_up, at_drop, delivered, exception, cancelled`	dispatch functions (up to `assigned` / `cancelled`); the rest by the rider app (not in repo)
`delivery_assignments.outcome`	`accepted, rejected, timed_out, withdrawn`	dispatch functions
`ol_orders.payment_status` / `ol_checkouts.payment_status`	`not_charged, pending, paid, failed, refunded`	payments code / triggers / refund code
`ol_refunds.status`	`requested, approved, rejected, processed`	only `processed` is ever written
`ol_stores.status`	`draft, pending, live, paused`	admin API
`riders.status`	`draft, submitted, verification_pending, approved, rejected, documents_expired, suspended`	by hand (no code writes it)
`ol_booking_enquiries.status`	free text, default `demo_enquiry_saved`	–
`ol_support_requests.status`	free text, default `saved_for_review`	–

Specific conflicts: 1. **"Out for delivery" differs:** `ol_partner_act('out')` sets `ol_orders.status='out'` and `kot_tickets.status='delivered'` but leaves POS `orders.status='ready'`; `delivery_jobs` has its own `picked_up/at_drop`. 2. **"Delivered" can be set in three independent places** (`ol_orders` by staff, `orders` by staff via the same RPC, `delivery_jobs` by the rider app); no code reconciles them. 3. **Cancelled** can be set by restaurant reject/timeout (`ol_orders`, `orders`) and separately by `dispatch_cancel` (`delivery_jobs`); the job is cancelled by the sweep only after `ol_orders` is cancelled; the reverse (job cancelled → order cancelled) does not exist. 4. `ol_orders.status` for customer-checkout orders has no writer after `placed` (INFERRED); the customer UI ignores it and runs a timer. 5. `payment_status 'paid'` for a POS-linked cash order is set at staff "delivered" time, while for a customer-checkout COD order it stays `not_charged` forever (INFERRED; nothing marks COD collected). 6. `ol_orders.payment_method` CHECK was widened in 0025; the customer app type `PaymentMethod` in `frontend/src/customer.tsx` lacks `emi` / `cardless_emi` (these are only reported back by Razorpay). 7. `ol_refunds.method 'wallet'` exists though no wallet exists.

3. Different ID formats

Concept	Formats in use
Order number shown to humans	POS `display_no` `OL-00012` (per-org counter); dispatch `order_no` `OL-3F9A1B` (first 6 hex of the order uuid); Razorpay description `Order #3F9A1B2C` (8 hex of the **checkout** uuid) — the same `OL-` prefix, two meanings
Store / product / category / brand / banner / event / reel ids	`text` (`s1`, `p4`, `c1`, `org-<slug>-<outlet>`, `<store>_<uuid>_<uuid>`)
Organization, location, user, order, checkout, job, rider	`uuid`
`delivery_jobs.pos_order_id`	`uuid` with **no foreign key** (every other link to `orders` has one)
`rider_store_links.store_id`, `delivery_jobs.store_id`, `ol_orders.store_id`	`text` references `ol_stores(id)`
City	`city_id text` (`latur`) vs outlet `location_id uuid` vs address lat/lng
Customer ownership	64-hex hash text vs `user_id uuid`
Keys	device key hex vs idempotency key `^[A-Za-z0-9_-]{8,128}$` vs dispatch `idempotency_key` `onelocal:<uuid>:delivery` vs POS `external_ref` `onelocal:<uuid>`

4. Inconsistent naming

- Product/brand names: **OneCity** (app display name, storage keys `onecity.*`, bucket `onecity-catalog`, Live Activity module `onecity-live-activity`), **One Latur** (docs, `onelatur.*` keys, `/api` index message "One Latur API", rider app id `onlatur-rider`), **One Local** (tables `ol_*`, delivery app id `one-local`), **VINII** (POS/Partner), **Proviyaa** (OS/Master).
- Table prefixes: `ol_*` everywhere except the 0029 delivery tables (`riders`, `rider_store_links`, `rider_availability`, `push_tokens`, `org_dispatch_settings`, `delivery_*`) whose generic names live in the same `public` schema as other apps.
- `customer_hash` (database column) vs `customer` (same value in the in-memory types of `store.ts`).
- `orders` (POS) vs `ol_orders` vs the route `/api/orders` (= `ol_orders`) vs `/api/partner/orders/:id` (= POS order id) — the same word with two id spaces. The Partner `id` is the **POS order id**, `one_local_id` is the `ol_orders.id`.
- Statuses spelled `cancelled` (UK) in all tables; `canceled` not used.

5. Multiple sources of truth (summary)

Fact	Competing sources
Is a city live?	`ol_service_areas` vs `catalog.ts` constants
Price of an item	POS `menu_items.price` / `menu_item_variants.price` (marketplace) vs `ol_products.price` (demo) vs the stored `ol_orders.items[].price` (snapshot)
Is an item available	`menu_items.is_available`, `item_channel_status`, `ol_products.is_available`, catalogue cache (30 s)
Where to pick up	`ol_stores.lat/lng`, `ol_merchant_channels.lat/lng/address`, `locations.address` (admin preview uses channel, then location address)
Has the order been paid	`ol_checkouts.payment_status`, `ol_orders.payment_status`, Razorpay (webhook is the declared truth), POS `orders.paid_at` / `payments`
Where is the order / rider	`ol_orders.status`, `orders.status`, `delivery_jobs.status`, `delivery_events`, client timer, `ol_orders.rider_name` placeholder
Who is the customer	`auth.users`, `customer_hash`, POS free-text name/phone, `delivery_jobs.customer_*`
Consent state	phone AsyncStorage `onecity.privacy.v1` vs `ol_consents` (0027 DRAFT)

Fact	Competing sources
Migration numbering	repo files `0025 – 0029` are placeholders; the OS ledger (`schema_migrations` , Repo-1-) assigns real numbers. `docs/PLATFORM_FOUNDATIONS.md` ("New tables this needs (migrations 0025–0026 in Repo-1-)") reserves 0025–0026 for **different** planned tables: `apps` , `shoppers` , `ol_addresses` , `riders` , `platform_admin_scopes` , `app_devices` (generalises `ol_partner_devices` , "merged later"), `app_config` , `feature_flags` , `audit_log` . None exist in this repo's SQL. The same docs plan a `riders` table to come from the rider app, while this repo's draft 0029 also creates `riders` (**possible collision**, UNKNOWN which one the OS ledger will carry)

6. Other inconsistencies noticed while mapping

- `ol_order_items.product_id` FK vs marketplace product ids (INFERRED failing insert; see #3 above).
- `ol_coupons` has no redemption table or per-user limit: a code can be applied on every checkout (no table records use).
- `ol_orders.cancelled_by` , `ol_booking_enquiries.resolved_by` , `ol_support_requests.resolved_by/resolved_at` , `ol_order_items.discount` , `ol_media_assets` have no writer in this repo.
- `ol_partner_act('out')` and `('delivered')` are not followed by any dispatch call; `onRestaurantAccepted/Rejected` are the only hooks.
- The Partner `delivered` action marks cash collected on the POS (`payments`) even when a rider (not the restaurant) collected the cash; `delivery_jobs.cash_to_collect` is tracked separately and no reconciliation exists.
- Journal 2026-09-27 says "Migrations 0021–0024 are confirmed live in production" while `docs/CENTRAL_DB_MAP.md` (2026-09-22) still lists the superseded first-draft `ol_*` tables; the map is stale.
- `catalog.ts` header says "After the founder's clean-up (db/cleanup/) no demo rows are left", yet the JSON fallback (`data/catalog.json`) still serves demo data when no database is configured.

11 — UNKNOWN OR UNVERIFIED (One Local / OL-ui)

Everything below could not be established from this repository alone. Items are grouped by why. "DOC-CLAIM" = stated in `docs/` or `docs/JOURNAL.md` but not provable from code or SQL.

1. Live database state (this audit read files only; no database was queried)

#	Unknown	What the repo says
U1	Which migrations are actually applied to production project `ywylhzcxuoeilygzdonx`	DOC-CLAIM (journal 2026-09-27): 0021–0024 live; 0025–0029 not applied (0025/0026 on the local DB; 0029 tried locally and removed). Verify with `python3 scripts/db.py status` (read-only)
U2	Current production row counts (orders, listings, riders, etc.)	Journal: demo catalogue removed 2026-09-27; 9 organizations in OS (`CENTRAL_DB_MAP.md` 2026-09-22 snapshot: 9 organizations, 41 POS orders, 26 `org_users`). Not re-checked
U3	Whether `ol_orders` in production contains orders with `organization_id` / `pos_order_id` (i.e. any P2-path orders other than Partner test orders)	–
U4	Whether `ol_order_items` has any rows in production (INFERRED FK failure would leave it empty)	–
U5	RLS **policies** on all OS/POS tables and on `platform_admins` , `locations` , `menu_item_variants`	Counts only in `CENTRAL_DB_MAP.md` (140 total); the local replica has none
U6	Storage policies on `storage.objects` for `onecity-catalog`	none defined here
U7	Whether the production `orders.status` / `payments.method` CHECK lists equal the local replica's	replica is "generated from production's catalog" (header) as of its generation date
U8	Production values of Postgres `realtime` publication membership	not configured in this repo
U9	Whether `locations` has rows and whether Master-created listings use `location_id`	snapshot: 0 rows
U10	Exact Supabase Auth providers enabled (phone OTP SMS provider, Google OAuth, email confirmation)	the app asks `GET /auth/v1/settings` at runtime; local `config.toml` defaults only

2. Behaviour outside this repository

#	Unknown	Notes
U11	The **Rider app** (One Rider / One Local Delivery): how it handles pickup, drop, proof, OTP verification, exceptions, cash; who sets `delivery_jobs.status` beyond `assigned` ; how riders are created/approved	Only the pool-and-offer half is in this repo (`rider.ts`); docs `PRD_ONLATUR_RIDER_V1.md` , `RIDER_APP_E2E_V1.md` , `RIDER_DISPATCH_API.md` describe the rest
U12	The **POS (VINII, Flutter/web)**: how it consumes One Local orders, whether it reads `external_ref` , whether it handles `placed_by='integration'` , `channel='online'` ; existence in production of `pos_place_order` , `pos_set_order_status` , `pos_mark_handed_over` (named in `PLATFORM_FOUNDATIONS.md` §6)	not in this repo or its replica
U13	The **Master (Repo-1-)**: whether it already calls `/api/admin/*` , what it writes directly, whether it reads `ol_*` tables	`MASTER_CONTRACT_V1.md` , `MASTER_ADMIN_HANDSHAKE.md`
U14	**Who calls** `POST /api/dispatch/sweep` and how often; who holds `DISPATCH_SERVICE_TOKEN` / `DISPATCH_SIGNING_SECRET` ; whether dispatch is configured in any deployed environment	journal: "A scheduler must call the sweep every 15 to 30 seconds once this is live"
U15	Any scheduler for privacy retention purge	journal: "not built"
U16	How a customer-checkout food order would be routed to a restaurant (which function, which key mapping product id → `menu_item_id`)	documented as a future `ol_place_checkout` → `pos_place_order` (build-order C5); not in code
U17	Who accepts / progresses a customer-checkout order that has no `organization_id` (e.g. grocery/shop stores)	no code path found
U18	Whether the customer is ever shown the delivery code	`deliveryOtp()` has no caller outside `dispatch.ts`

#	Unknown	Notes
U19	Which of the two "order stage" truths the customer should see in future	UI uses a timer today
U20	Where the Expo API server is actually deployed (`BACKEND_URL` is an env var). Docs name Vercel as the production host (`docs/MASTER_ADMIN_HANDSHAKE.md` , `docs/DPDP.md` : "Vercel or a container host"); `.emergent/*` shows an Emergent preview-pod setup. Not confirmed	`docs/` , `.emergent/*` , `app.config.ts`
U21	`EAS_PROJECT_ID` / OTA update channel in use	env
U22	Which screens call the SecureStore wrapper (`frontend/src/utils/storage/index.ts`)	not traced
U23	Whether `backend/` (FastAPI media store + seed scripts) is still deployed anywhere	no server entry file in repo; `/api/media/[id]` replaces it
U24	Pricing items the founder has not decided: handling fee, platform fee, GST on delivery, GST rate per category (CA), rider payout per delivery	`pricing.ts` constants, `docs/JOURNAL.md` open questions
U25	Refund policy and the money-movement path (cash handback, Razorpay refund API)	`0024` header: "Phase 3 concern"

3. INFERRED statements (deduced from code, not written down)

#	Inference	Basis
I1	Customer-checkout orders never reach the Partner app or the POS	`createCheckout` sets no `organization_id` / `pos_order_id` ; `restaurant.ts` filters on `organization_id` ; journal admits "not yet connected"
I2	`ol_orders.status` of a customer-checkout order stays `placed` indefinitely	the only writers of `ol_orders.status` are `ol_partner_act` and the checkout insert
I3	`ol_order_items` inserts fail for marketplace products	FK `product_id → ol_products(id)` ; marketplace ids are synthetic `<store>_<uuid>` strings absent from `ol_products` ; `ol_products` emptied by the clean-up
I4	A COD checkout order stays `payment_status = 'not_charged'` forever	no code marks cash collected for non-POS-linked orders
I5	`ol_expire_unanswered` is the only timeout trigger and depends on a Partner app polling the list	called only from `restaurant.ts listOrders`
I6	A blocked shopper can keep using an existing access token for up to ~60 s	`auth.ts` cache TTL
I7	`ol_refunds.status` `requested/approved/rejected` are never written	no writer
I8	`backend/` Python is legacy	no server file; `BACKEND_REARCHITECTURE_PLAN.md`
I9	Possible `riders` table collision with the rider app's planned table	docs vs 0029
I10	Two simultaneous servers would each hold separate rate-limit buckets and catalogue caches	in-process Maps
I11	A customer order without `user_id` (device-only) has no name/phone on the delivery job	`createJobForOrder` reads user metadata only when `o.user_id`

4. Not examined

- Binary/large assets (`frontend/assets` , `Sources/` , `backups/`) and `node_modules` .
- The content of `frontend/server/data/catalog.json` (4,092 lines, demo seed), `india-places.tsv.gz` , `media.json` beyond their role.
- `frontend/e2e/*` , `test_reports/*` , `tests/` , `design_guidelines.json` , `image_testing.md` — test/design material.
- `docs/*.pdf` , `docs/research/*` , `docs/ONELOCAL_COINS_PLAN.md` , `docs/SWIGGY_*` reference/planning docs were not read in full; only headings of `PLATFORM_FOUNDATIONS.md` and the journal were used.
- Git history other than the status given at session start.
- The `ios/` native project and `frontend/targets/widget` Swift beyond their names.
- `.env` , `.env.local` contents were deliberately **not** opened (secrets). Variable names come from code.

12 — APPLICATION SUMMARY (One Local / OL-ui)

Forensic extraction of THIS application only. Detail and evidence are in files 01–11. Tags: no tag = stated in the repo; **INFERRED**; **UNKNOWN**; **DRAFT** = migration not applied to production per `docs/JOURNAL.md`.

APPLICATION: `OL-ui` repository, git branch `feat/expo-router-api` (audit date 2026-10-02). Contains three deployable pieces: (1) the customer app + its API server (`frontend/`, Expo Router; app display name "OneCity", also branded "One Latur" / "One Local"), (2) "VINII Partner", the restaurant order-accepting app (`partner/`), (3) database migrations and tooling (`db/`, `scripts/db.py`, `supabase/`). A legacy Python `backend/` (media + seed scripts) remains; `docs/` holds contracts and the development journal.

PURPOSE: A hyperlocal commerce marketplace for the city of Latur (Maharashtra): customers order food from restaurants and shop groceries, shops, pharmacy and beauty items from local stores, pay by cash on delivery or online (Razorpay), and a rider pool is pinged to deliver. Restaurants run on the VINII POS (shared database) and accept One Local orders in the Partner app. Outside Latur the app offers a "discover" mode (nearby places, no ordering) and collects "launch here" demand.

TECH STACK: Expo SDK 57 / React Native 0.86 / React 19, Expo Router (file routes + `app/api/**/+api.ts` server routes), TypeScript, TanStack Query, `@supabase/supabase-js` 2.116, AsyncStorage, `react-native-webview` (Razorpay Checkout), `expo-notifications`, iOS Live Activity/widget (Swift), `react-native-maps` / `@vis.gl/react-google-maps`, Playwright e2e. Partner app: Expo/React Native. Server logic: `frontend/server/*.ts`. Database: Supabase Postgres 17 (SQL migrations, PL/pgSQL RPCs). Legacy: Python/FastAPI scripts in `backend/`.

DATABASE: Supabase project `ywylhzcxuoeilygzdonx` ("VinniPos", Mumbai) shared with the OS/Master and the POS; local dev project `onelocal-dev`. This repo owns the `ol_*` tables (23 tables incl. DRAFT, plus the view `ol_consent_current`) and 8 delivery tables (`riders`, `rider_store_links`, `rider_availability`, `push_tokens`, `org_dispatch_settings`, `delivery_jobs`, `delivery_assignments`, `delivery_events`) and 1 view `delivery_orders_needing_rider`. Reads/writes OS/POS tables (`organizations`, `org_users`, `roles`, `menu_*`, `item_channel_status`, `orders`, `order_items`, `order_events`, `kot_tickets`, `payments`, `platform_admins`, `locations`). All via service role; RLS ON with **no policies** on every `ol_*` table (see 06). Migrations 0021–0024 live (DOC-CLAIM); 0025 payments, 0026 checkouts, 0027 privacy, 0028 listings, 0029 dispatch are **DRAFT**; the server detects missing pieces and degrades (COD only, no checkout row, consents on phone, listing API 503, dispatch off).

AUTH: Supabase Auth, one login for all apps. Customer: email+password, phone OTP, Google (PKCE) — **plus an anonymous device key** (`x-customer-key`) the server accepts for checkout; Partner: email+password then `org_users` membership + role; Rider: phone OTP + approved `riders` row; Master/admin: `platform_admins` row + CORS allow-list; service-to-service dispatch: bearer token + HMAC signature; Razorpay: webhook HMAC. Authorization is entirely in application code.

PRIMARY USERS: Customers (shoppers) in Latur; restaurant staff (VINII Partner); riders (separate app, only dispatch API here); Proviyaa platform admins (Master console calling `/api/admin/*`); POS operators indirectly (shared tables); the founder/operations (journal, demand report).

ORGANIZATION MODEL: OS `organizations` (uuid) own POS menus and staff (`org_users` + `roles`). One Local links an organization to a marketplace listing `ol_stores` (`organization_id`, optional `location_id`) approved by a platform admin (`status draft/pending/live/paused`; one live listing per organization+outlet). `ol_merchant_channels` holds one row per organization (accepting switch, accept window, prep time, pickup address). Orders and data are scoped by `organization_id` (merchant) and `city_id` (market) — but customer-checkout orders carry no `organization_id` (see Risks).

LOCATION MODEL: `city_id text` (`latur`) via `ol_service_areas` (centre + radius; DB for coverage, **hard-coded constants for ordering/dispatch/rider checks**). Outlets are OS `locations` (uuid), referenced only by `ol_stores.location_id` (draft). Delivery points are lat/lng inside the address JSON; pickup point is `ol_stores.lat/lng` or `ol_merchant_channels.lat/lng`. Distance by haversine; no PostGIS, no zones. India-wide "discover" uses a bundled places file.

ORDER MODEL: Customer side: a **checkout** (`ol_checkouts`, DRAFT) = one payment; **one** `ol_orders` **row per store** in the cart (food cart = one restaurant; shopping cart = many stores; the two never mix). Lines: jsonb `ol_orders.items` (read path) + `ol_order_items` (normalized, best effort). Status `placed → accepted → packed → out → delivered | cancelled` (`ol_orders.status`), advanced only by the Partner state machine `ol_partner_act` on POS-linked orders. Kitchen side: POS `orders` (`new → sent_to_kitchen → ready → delivered | cancelled`), `kot_tickets`, `order_events`. Two creation paths exist and are not joined: customer checkout (no POS order) vs `ol_create_restaurant_order` (POS-linked, used only by the Partner test order). IDs are uuids generated in the server (`crypto.randomUUID()`); idempotency via `Idempotency-Key` + unique index.

PAYMENT MODEL: Cash on delivery (default) or Razorpay (UPI/card/netbanking/wallet/pay-later; EMI reported back). One Razorpay order per checkout for the server-computed total; payment state lives on `ol_checkouts` and is copied by trigger to its orders (DRAFT 0026; before it, online is disabled). Confirmation by app `verify` (signature) and by webhook (source of truth); webhook log `ol_payment_events`; refunds recorded when Razorpay reports them (`ol_refunds`); an internal refund function exists but nothing calls it. POS cash is recorded in POS `payments` only when staff mark a POS-linked order delivered. No wallet, coins, invoices; COD collection for non-POS orders is not recorded.

INVENTORY MODEL: None (no stock quantities, no decrement). Availability flags only: `menu_items.is_available`, per-channel switch `item_channel_status` (`channel='one_local'`, `off_until`) edited in the Partner app, `ol_products.is_available` (demo). Catalogue cached 30 s.

DELIVERY MODEL: Own rider pool (DRAFT 0029): a `delivery_jobs` row per successful order (COD at placement; online when paid), offered to one rider at a time (default 30 s, 3 rounds, 8 km, linked/nearest/fewest-jobs-today), first accept wins (database-enforced), escalation to the restaurant when no taker; sweep + heartbeats drive timeouts; rider privacy rules; delivery code derived from the order but never shown or verified (INFERRED unfinished). Job states after `assigned` belong to the rider app (not in repo). Customer tracking screen is a **client-side timer** with "Sample rider". The restaurant's own "out/delivered" staff buttons are an independent status path.

OFFLINE MODEL: No offline support. AsyncStorage only (cart, profile/addresses, device key, consents, wishlist). No SQLite, outbox, queue or sync. Retries are manual or idempotent re-sends; server-side idempotency and optimistic version checks prevent duplicates.

MAIN TABLES: `ol_orders`, `ol_checkouts` (DRAFT), `ol_order_items`, `ol_refunds`, `ol_payment_events` (DRAFT), `ol_stores`, `ol_products` (demo), `ol_merchant_channels`, `ol_partner_devices`, `ol_coupons`, `ol_service_areas`, `ol_categories`, `ol_brands`, `ol_banners`, `ol_reels`, `ol_events`, `ol_themed_stores`, `ol_discover_chips`, `ol_booking_enquiries`, `ol_support_requests`, `ol_consents` / `ol_privacy_requests` (DRAFT), delivery tables (DRAFT); borrowed: `organizations`, `locations`, `org_users`, `roles`, `platform_admins`, `menu_categories`, `menu_items`, `menu_item_variants`, `item_channel_status`, `orders`, `order_items`, `order_events`, `kot_tickets`, `payments`, `org_counters`, `auth.users`.

MAIN APIs: Customer: `POST /api/cart/quote`, `POST /api/checkouts`, `GET /api/orders[/:id]`, `GET /api/checkouts/:id`, `POST /api/payments/order|verify|cod|webhook`, `GET /api/catalog|stores|products|coverage|nearby|places/*`, privacy `/api/privacy/*`. Partner: `/api/partner/me|orders|orders/:id/{accept,reject,ready,out,delivered}|outlet|items|devices|test-order`. Rider: `/api/rider/availability|heartbeat|push-token|offers/current|offers/:id/{accept,decline}|jobs/*`. Service: `/api/dispatch/jobs[...]|sweep`. Master: `/api/admin/me|organizations|listings[...]|shoppers[...]|media/upload-url|demand`. RPCs: `ol_create_restaurant_order`, `ol_partner_act`, `ol_expire_unanswered`, `ol_process_refund`, `dispatch_create_job|release|offer_next|answer|cancel|due_jobs`, `next_org_seq`.

MAIN EVENTS: No realtime, queue, DB webhook or edge function. Event sources: `delivery_events` outbox (`delivery.created|released|offered|timed_out|no_rider|assigned|rejected|cancelled`), POS `order_events` (`placed, accepted, kot_fired, rejected, kot_ready, out_for_delivery, delivered, paid, timed_out`), Razorpay webhooks (`payment.captured, order.paid, payment.failed, refund.processed`), Expo push (`new_order, no_rider, job_offer, job_assigned, job_cancelled`), DB trigger `ol_checkouts_payment_to_orders`. Polling: Partner 5 s/10 s, rider heartbeat 30 s, dispatch sweep (caller UNKNOWN).

SOURCE-OF-TRUTH ENTITIES: Identity: Supabase `auth.users`. Merchant, outlet, staff, menu, kitchen order, KOT, POS payment: OS/POS tables. Customer order, checkout and payment state, listing, merchant channel, coupon, service area, refund ledger, consent, privacy request: One Local `ol_*`. Delivery job, offers, rider presence, delivery events: 0029 tables (rider app owns later states). Money: Razorpay.

KNOWN DUPLICATIONS: `ol_orders` vs POS `orders` (by design, but unlinked for customer checkouts); `ol_orders.items` vs `ol_order_items`; payment fields on `ol_checkouts` and `ol_orders`; `ol_products` vs POS items projected into products; `ol_merchant_channels` vs POS `channel_integrations`; pickup point in `ol_stores` and `ol_merchant_channels`; `ol_partner_devices` vs `push_tokens`; `ol_service_areas` vs hard-coded city constants; fee/GST rules in TypeScript, SQL and (ignored) POS bill settings; three order-number formats sharing the `OL-` prefix; four competing order-progress sources; `ol_support_requests` used for three purposes. Full list in 10.

KNOWN RISKS: 1. Customer food orders never reach the restaurant/POS (no `organization_id` / `pos_order_id`; Partner app lists only org-linked orders) — journal agrees. 2. `ol_order_items` insert likely fails for marketplace products (FK to `ol_products`) — INFERRED; refunds and analytics rely on it. 3. `ol_orders.status` never advances for customer-checkout orders; COD never marked collected; tracking is simulated. 4. Dispatch: depends on unapplied 0029 and secrets, on an unknown scheduler, and stops at `assigned`; delivery code unused; cash on delivery is tracked on `delivery_jobs` but not reconciled. 5. Service-role key for all access, RLS gives no backstop; per-process rate limiting; server accepts device-only checkouts. 6. Non-atomic checkout (checkout row, orders, lines, dispatch are separate calls); a restaurant order timeout needs Partner polling. 7. Five migrations are drafts with placeholder numbers; possible collision with the OS ledger / rider app's `riders` table. 8. No coupon redemption tracking; no customer cancel/refund request flow; refund RPC has no caller. 9. Stale docs: `CENTRAL_DB_MAP.md` pre-dates 0021. 10. DPDP: POS keeps its own copy of customer name/phone outside erasure reach (journal).

UNKNOWN AREAS: Live production schema/rows/policies for OS/POS tables; deployed hosting and who runs the dispatch sweep; rider-app behaviour after assignment; POS-side consumption of One Local orders and existence of `pos_*` functions in production; Master's actual use of `/api/admin/*`; Auth provider settings (SMS/Google); exact production state of migrations 0025–0029. Full list in 11.